Generic Data Mart (CDP) Pipeline

Technical Architecture

Metadata-driven orchestration : DBT (Docker) build : Java Dataflow transformation

Document	Technical Architecture & Build Specification
Audience	Solution architects, developers, testers, engineering leads
Author	Joseph Aruja (Senior Lead Software Engineer)
Version	1.0 — first issue (for review)
Status	For review & build sign-off
Date	June 2026
Pattern	Contract-driven ingestion : Monthly batch (single run)

Two stages per Data Mart per month — **CDP_BUILD** then **TRANSFORMATION** — gated entirely by a single metadata control table. No direct job-to-job chaining.

Contents

Executive Summary

This document describes the architecture for the monthly Data Mart pipeline, together with the rationale behind each decision. Curated **Feature/Data Products (FDPs)** are the sole input layer. Multiple FDPs feed a single **Data Mart (CDP)**, which **DBT builds in Docker**. The same architecture builds **any CDP**; where a CDP needs a downstream file (e.g. for Mainframe re-ingestion), an **optional Java Dataflow transformation** reads the mart, performs its heavier shaping in a **BigQuery processing dataset**, and writes the **final CSV output to Cloud Storage**. CDPs that do not need that are consumed directly from the Data Mart.

The central design principle is that **no job invokes another job directly.** An enterprise scheduler (Tivoli Workload Scheduler / TWS) drives all execution, and a single **metadata control store** — a *transactional* database (Cloud SQL/Spanner), not BigQuery — is the source of truth for FDP readiness and stage status, modelled as a parent stage table and an FDP-source child table. Execution advances only when the metadata conditions are met, producing a fully auditable, restartable, and idempotent monthly run with a clean separation between **data flow** and **control flow**.

Up to two stages run per Data Mart per month, in strict order:

1. `CDP_BUILD` — DBT builds the Data Mart. **Always runs.**
2. `TRANSFORMATION` — Dataflow produces the downstream file. **Optional** — runs only for CDPs configured with `has_transformation` (Section 16.1).

When present, `TRANSFORMATION` starts **only** when `CDP_BUILD = COMPLETED` **and** `TRANSFORMATION = NOT_STARTED`. A build-only CDP completes after `CDP_BUILD`.

Within a CDP the stages are ordered (and transformation, if any, follows the build), but each CDP is independent and **runs in parallel**: the scheduler fans out across all CDPs for the month, with end-to-end isolation by `cdp_name` (Section 4.4). The platform is **Google Cloud** throughout.

---

1. Context & Scope

1.1 Background

The organisation needs to produce a number of **Data Marts (CDPs — Consumable Data Products)** on a **monthly** cycle, each one assembled from curated upstream data and, where required, shaped into a file for a downstream consumer (for example a Mainframe re-ingestion). Historically this kind of work tends to grow into a sprawl of bespoke, hand-wired jobs — one set per mart — that are hard to operate, reason about, and recover when something fails.

This architecture replaces that with a **single, generic pipeline** that can build **any CDP** from configuration rather than bespoke plumbing. The same orchestration, control model, build mechanism, and (optional) transformation serve every CDP; what differs per CDP is **config** — which curated sources it consumes, how its fields map, and whether it needs a downstream file. New CDPs are onboarded by adding configuration and a mart model, not by building a new pipeline.

Two design ideas underpin everything:

- **Contract-driven ingestion.** The pipeline reads **only** from curated **Feature/Data Products (FDPs)** — never from raw source systems. Each FDP is a governed product with a stable contract (schema, partitioning, freshness, readiness), so the mart depends on *stable agreements* rather than volatile sources, and upstream teams can change their internals without breaking us.
- **Metadata-driven orchestration.** No job calls another job directly. An enterprise scheduler (TWS) drives execution, and a **transactional control store** is the single source of truth for what is allowed to run and what has run. Every step advances only when the metadata says it may — which makes the whole monthly run auditable, restartable, and safe to run many CDPs in parallel.

1.2 Key terms

Term	Meaning
FDP	*Feature / Data Product* — a curated, contract-bound upstream dataset; the **only** input to the pipeline
CDP	*Consumable Data Product* — a **Data Mart** built from FDPs; the unit this pipeline produces
CDS	the downstream **pre-processing / consumer** the final file feeds (e.g. Mainframe re-ingestion)
Mart	the curated BigQuery dataset/table a CDP build produces (`mart.cdp_<name>`)

Term	Meaning
Contract-driven	input is taken only via curated FDPs with stable contracts, not raw sources
Metadata-driven	execution is gated by a control store, with no direct job-to-job chaining

1.3 At a glance

Aspect	Decision
Ingestion model	Contract-driven; FDPs are the only input layer
Source access	**No direct source system access** by the mart or transformation
Mart build	DBT, executed via Docker container
Downstream transform	**Optional per CDP** — Java Dataflow (Apache Beam) only when `has_transformation` is set; build-only CDPs skip it
Orchestrator	Enterprise scheduler (TWS) — the **only** orchestrator
Chaining	None direct; **metadata-driven** gating
Control store	**Transactional DB (Cloud SQL/Spanner)** — *not* BigQuery; ACID claim, enforced keys
Cadence	Monthly; one run per `(CDP, month)`. Serialized within a CDP; CDPs are **parallel-capable** — TWS runs anywhere from one-at-a-time to all-at-once via `max_concurrency` (Section 4.4)
Platform	**Google Cloud** — BigQuery, Dataflow, Cloud Storage, Cloud SQL, Artifact Registry, IAM/Workload Identity, VPC-SC
Output	Intermediate → BigQuery processing dataset; Final → CSV in Cloud Storage

1.4 What this document covers (and what it does not)

This is the **technical architecture and build specification** for the generic Data Mart (CDP) pipeline. It is deliberately broad — it carries the design *and* enough build detail for engineers to implement.

In scope (covered here):

- Orchestration model and metadata control plane — catalogue, registry, runtime tables, scheduler DAGs (Sections 4–5)
- Component responsibilities; partitioning/clustering; chunking (Sections 6–7)
- Build (dbt-in-Docker) and the **optional** transformation (Dataflow), with the mapping sheet, mapping template, sharding and the **Mainframe output contract** (Sections 13–14)
- Build-out specification — naming, concrete SQL, TWS jobs, parallel-execution & resource sizing (Sections 11–16)
- IAM/security at architecture level; **observability** (metrics, logs, dashboards, Dynatrace); idempotency/restart; DoD; phased plan (Sections 17–21)
- **Non-functional requirements** and **service levels (SLI/SLO/SLA)** (Sections 22–23)
- Risks & mitigations (Section 9)

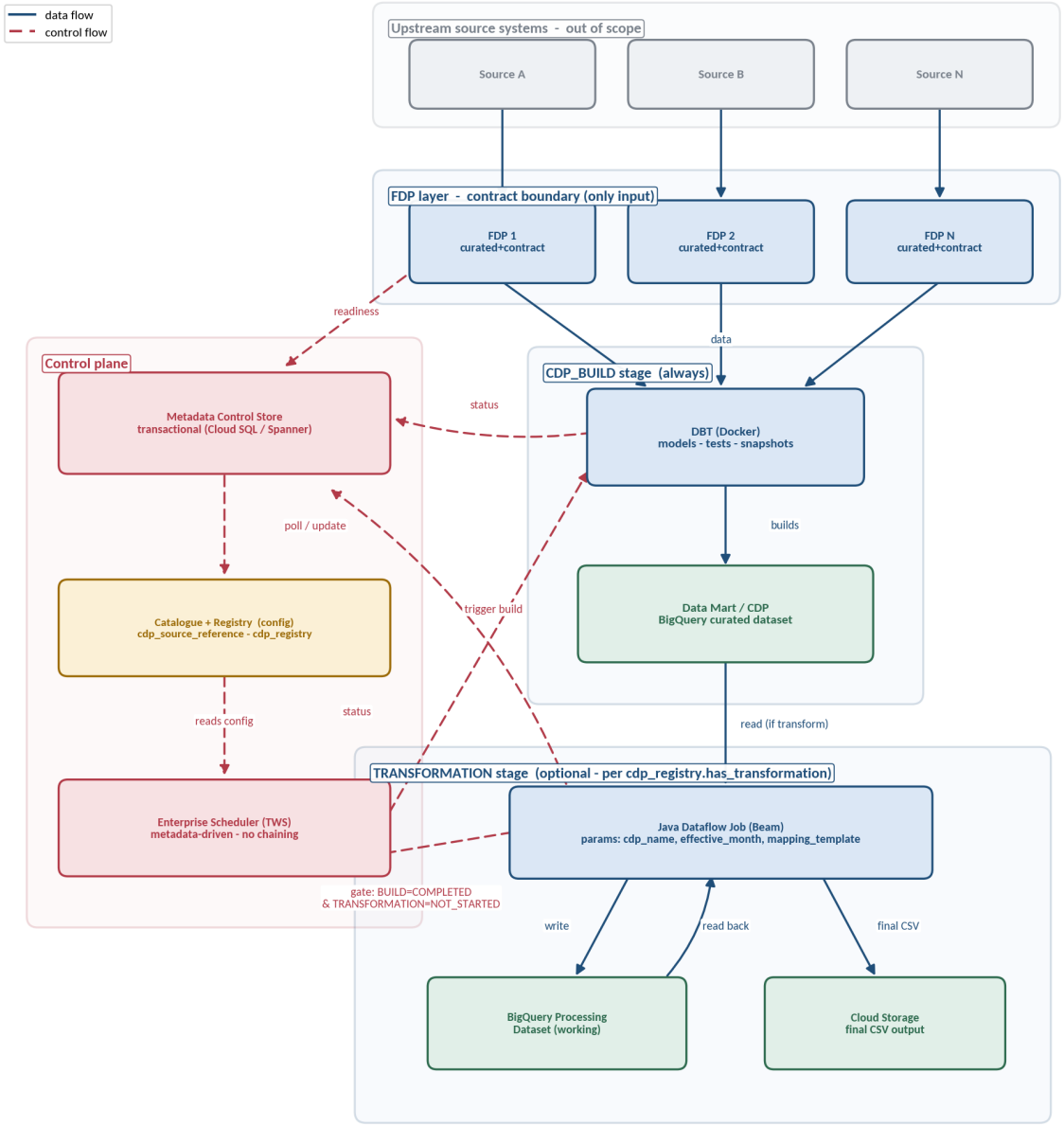
Out of scope / deferred (covered elsewhere or later):

- **FDP internal construction** — owned by upstream FDP teams under contract (this pipeline only consumes them)
- **Downstream consumption** and the **Mainframe-side conversion/ingestion** of the CSV (we deliver to GCS per the Output File Contract; the Mainframe converts and loads)
- **Infrastructure provisioning (IaC/Terraform)** and the **CI/CD pipeline implementation** — assumed to exist; this doc states the requirements (the Infrastructure Discussion Checklist is the companion)
- **PII & access-governance detail** — high-level only here (Section 24, *placeholder*); to be detailed with the security/governance team
- **Concrete volumetrics, SLO/SLA targets, and cost figures** — ***TBD*** (Sections 22.1, 23), to be supplied
- **Operational procedures** and the **support model** — see the companion **Run Book**; the **test strategy** — see the **Testing Architecture**

---

2. Detailed Architecture Diagram

The diagram separates **data flow** (solid, how bytes move) from **control flow** (dashed, how execution is decided and triggered).

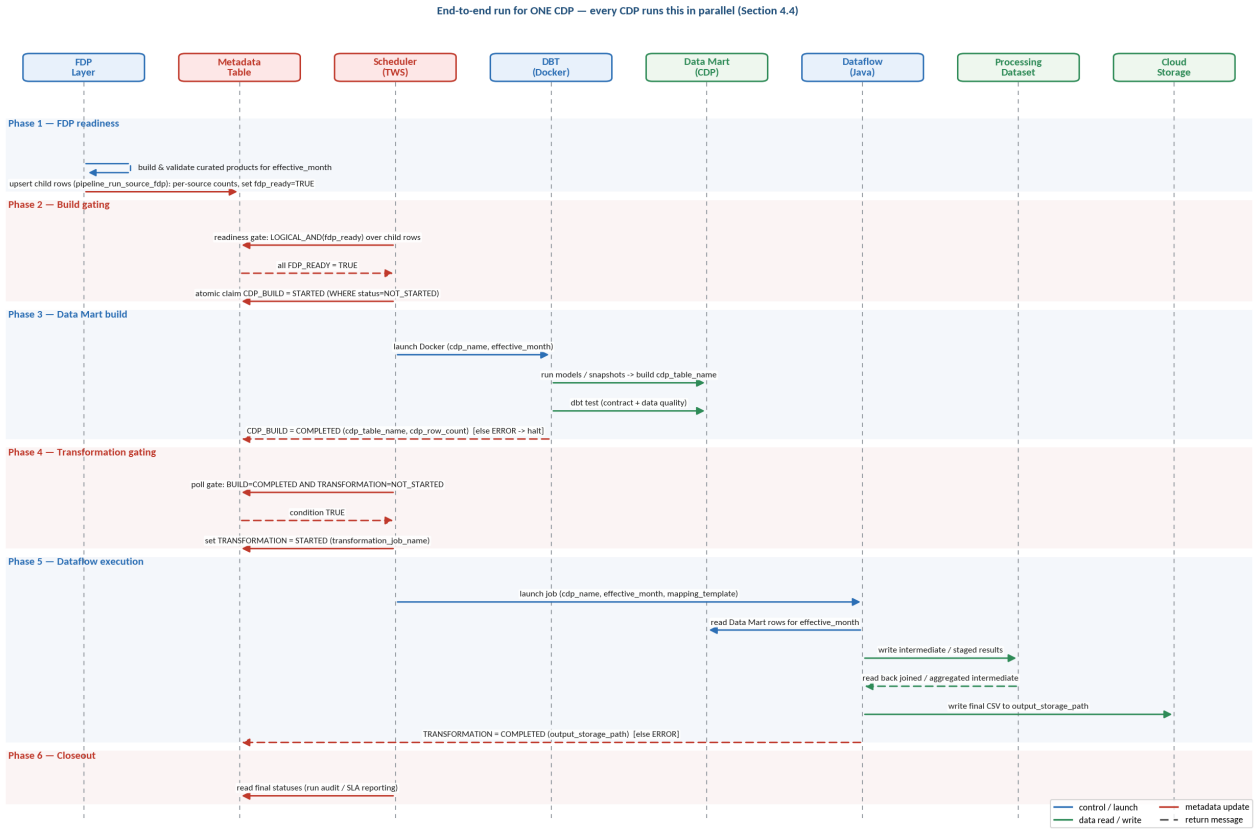


Reading the diagram

- **Solid arrows = data flow.** Source → FDP → DBT → Data Mart → Dataflow → Processing dataset → Cloud Storage. There is exactly one ingress path into the mart (FDP) and one egress path out of the transform (CSV).
- **Dashed arrows = control flow.** FDPs publish readiness to the metadata table; the scheduler polls metadata and is the only actor that triggers DBT and Dataflow; both stages write status back to metadata. No solid line ever skips the FDP layer, and no dashed line lets one job call another directly.

---

3. Sequence Diagram (End-to-End Monthly Run)



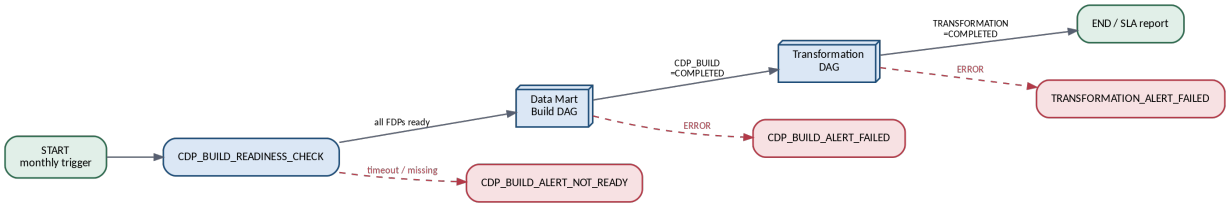
---

4. Scheduler DAG Design

The scheduler is the only orchestrator. Each "job" is a TWS job; dependencies are expressed as TWS conditional/predecessor links **plus** a metadata predicate that each job re-checks at runtime (belt-and-braces: the schedule order is advisory, the metadata gate is authoritative).

4.1 Master DAG (per `cdp_name`, monthly)

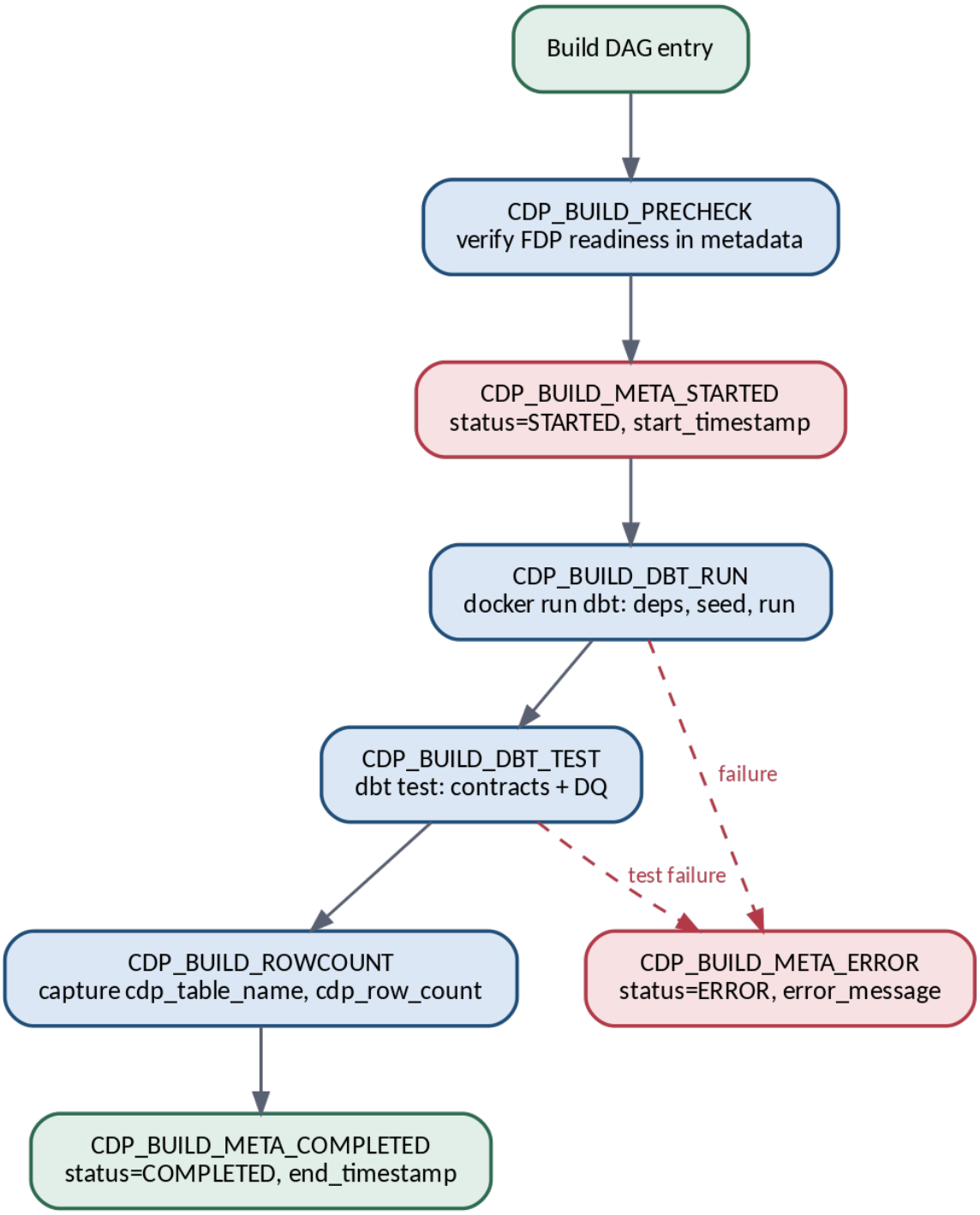
This is the cycle for **one** CDP. The driver (`CDP_FANOUT`, Section 4.4) instantiates it for every CDP **in parallel**. The **Transformation DAG runs only for CDPs with `has_transformation`** (Section 16.1); a **build-only CDP completes after the Build DAG** (no transformation).



Master DAG gating predicates

Edge	Metadata predicate (authoritative)
Readiness → Build	`LOGICAL_AND(fdp_ready)` over `pipeline_run_source_fdp` rows for `(cdp_name, effective_month)`
Build → Transformation	`stage=CDP_BUILD AND status=COMPLETED` **AND** `stage=TRANSFORMATION AND status=NOT_STARTED`
Any stage → Alert	`status=ERROR`

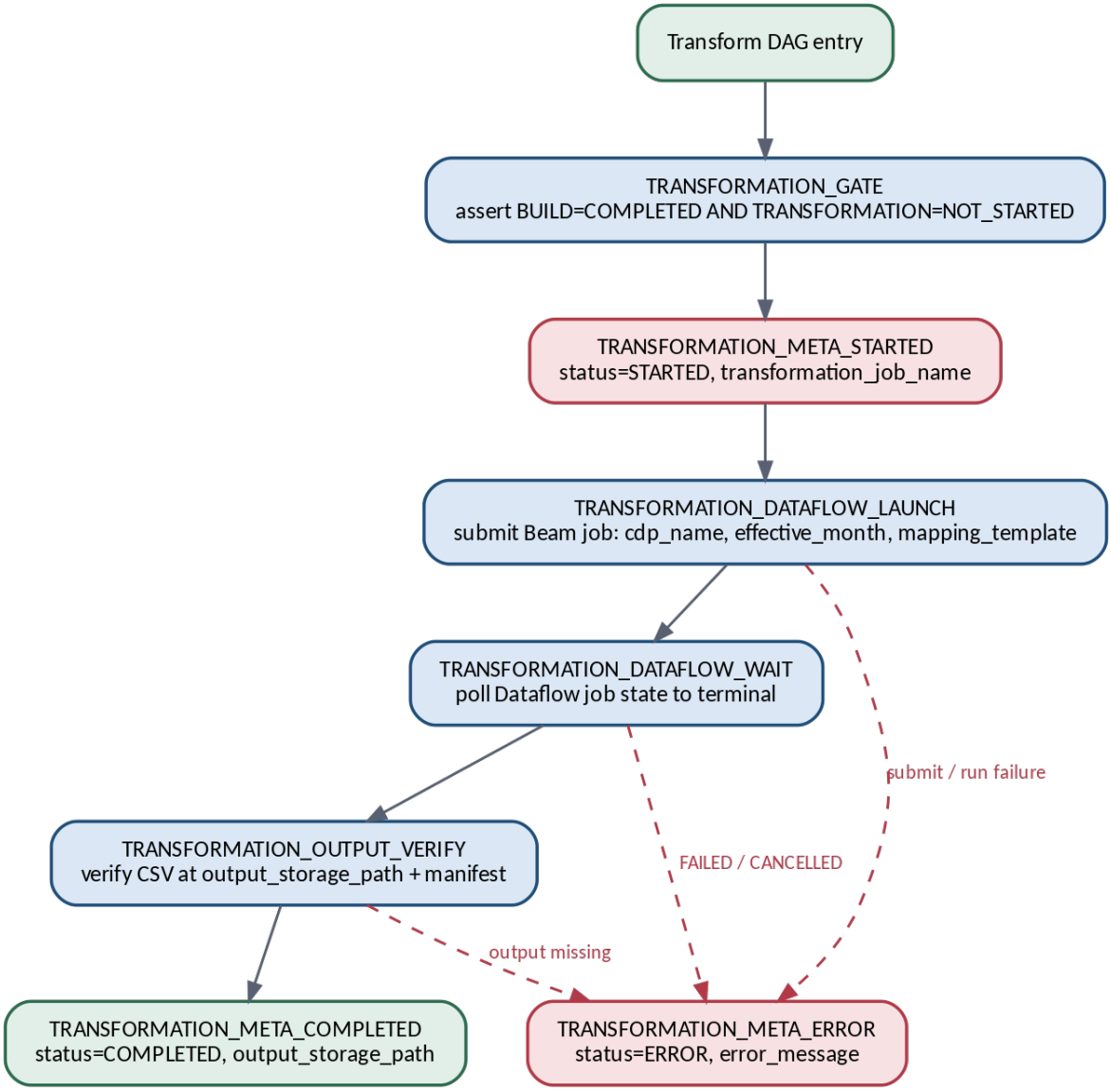
4.2 Data Mart Build DAG (`CDP_BUILD`)



Job	Responsibility	Success condition
`CDP_BUILD_PRECHECK`	Re-validate FDP readiness + idempotency (is this month already built?)	All FDPs ready, no prior COMPLETED
`CDP_BUILD_META_STARTED`	Set `CDP_BUILD` → STARTED	Row updated
`CDP_BUILD_DBT_RUN`	`docker run` DBT image; `dbt deps && dbt run` for the mart selector	Exit 0
`CDP_BUILD_DBT_TEST`	`dbt test` — schema/contract + data-quality tests	All tests pass
`CDP_BUILD_ROWCOUNT`	Query mart for `cdp_row_count`, capture `cdp_table_name`	Count returned

Job	Responsibility	Success condition
`CDP_BUILD_META_COMPLETED`	Set `COMPLETED`, write counts + `end_timestamp`	Row updated
`CDP_BUILD_META_ERROR`	On any failure: `ERROR` + `error_message`	—

4.3 Transformation DAG (`TRANSFORMATION`)



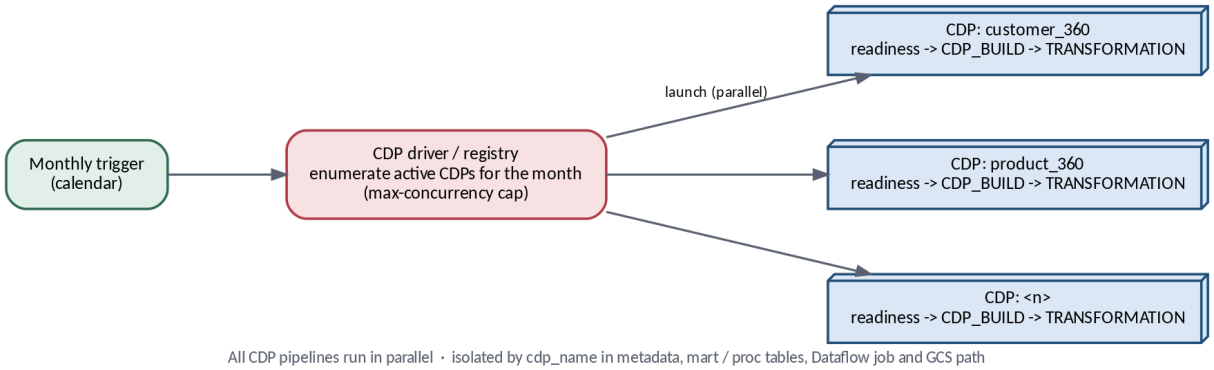
Job	Responsibility	Success condition
`TRANSFORMATION_GATE`	Enforce the trigger rule before doing anything	Predicate TRUE
`TRANSFORMATION_META_STARTED`	Set `STARTED`, record `transformation_job_name`	Row updated
`TRANSFORMATION_DATAFLOW_LAUNCH`	Submit Beam/Dataflow job with 3 params	Job accepted (job id)
`TRANSFORMATION_DATAFLOW_WAIT`	Poll Dataflow until `DONE/FAILED/CANCELLED`	State = DONE
`TRANSFORMATION_OUTPUT_VERIFY`	Confirm CSV(s) + row/manifest checks at path	Files present, counts reconcile
`TRANSFORMATION_META_COMPLETED`	Set `COMPLETED`, write `output_storage_path`	Row updated
`TRANSFORMATION_META_ERROR`	On failure: `ERROR` + `error_message`	—

Why a separate `WAIT` job: Dataflow submission returns immediately; the scheduler must treat job *completion* (not submission) as the gate for marking `COMPLETED`. Decoupling launch from wait also makes restart-after-launch safe.

4.4 Concurrency & parallelism — all CDPs run in parallel

The architecture is **parallel-safe by design**: every CDP is fully isolated, so any number of CDPs may run at the same time. How many *actually* run at once is a configuration choice, not a structural one. The driver's `max_concurrency` — together with TWS's own job-stream and workstation limits — sets the degree of parallelism anywhere on a spectrum: from **fully sequential** (`max_concurrency = 1`, where TWS runs the CDPs one after another) to **fully parallel** (all CDPs at once), or any wave size in between. The only *hard* serialization is within a single CDP's own run: two builds of the same `(cdp_name, effective_month)` never run concurrently, enforced by the atomic claim in Section 5.7.

This matters operationally: the build can **start sequential** (`max_concurrency = 1`, lowest resource footprint and easiest to support) and dial concurrency up as quota and confidence grow — with **no change to the architecture**, because isolation is already in place. TWS scheduling and the design are decoupled: the same design runs serially or in parallel.



The scheduler **fans out**: the monthly trigger drives a **CDP driver/registry** that enumerates the CDPs active for the month and instantiates the per-CDP Build→Transform DAG for each, running up to `max_concurrency` at a time (which may be 1 — i.e. strictly one after another). Per-CDP DAGs are independent — one CDP failing, erroring, or waiting on a late FDP does not block any other. Isolation is by `cdp_name` everywhere, which is what makes any concurrency level safe:

Concern	Isolation mechanism	Parallel-safe
Control state	Rows keyed by `(effective_month, cdp_name, stage)`; atomic per-row claim	Yes — per CDP
Mart tables	`mart.cdp_<name>`, partitioned by month	Yes
Processing dataset	Tables namespaced `proc.<cdp>_<YYYYMM>_<step>`, `WRITE_TRUNCATE`	Yes
Dataflow job	One job `df-<cdp>-<YYYYMM>` per CDP	Yes
Output	`gs://…/<cdp_name>/<YYYY-MM>/` per CDP	Yes
FDP readiness	Child rows scoped by `(effective_month, cdp_name)`	Yes
Shared SA / quotas	Single pipeline SA (shared); throttle via driver `max_concurrency`	Yes — within quota

Serialized vs parallel, precisely: *within* one CDP the two stages are strictly ordered (`CDP_BUILD` → `TRANSFORMATION`) and only one run exists per month. *Across* CDPs, execution is fully parallel. The only global constraints are **shared GCP quotas** — BigQuery slots (a dedicated reservation/assignment is recommended for the build window), Dataflow regional CPU and in-use-IP quota, and Cloud SQL `max_connections` — which the driver bounds with a configurable `max_concurrency` so that a high-CDP month cannot exhaust a project-wide limit.

5. Metadata Model — Catalogue & Control Plane

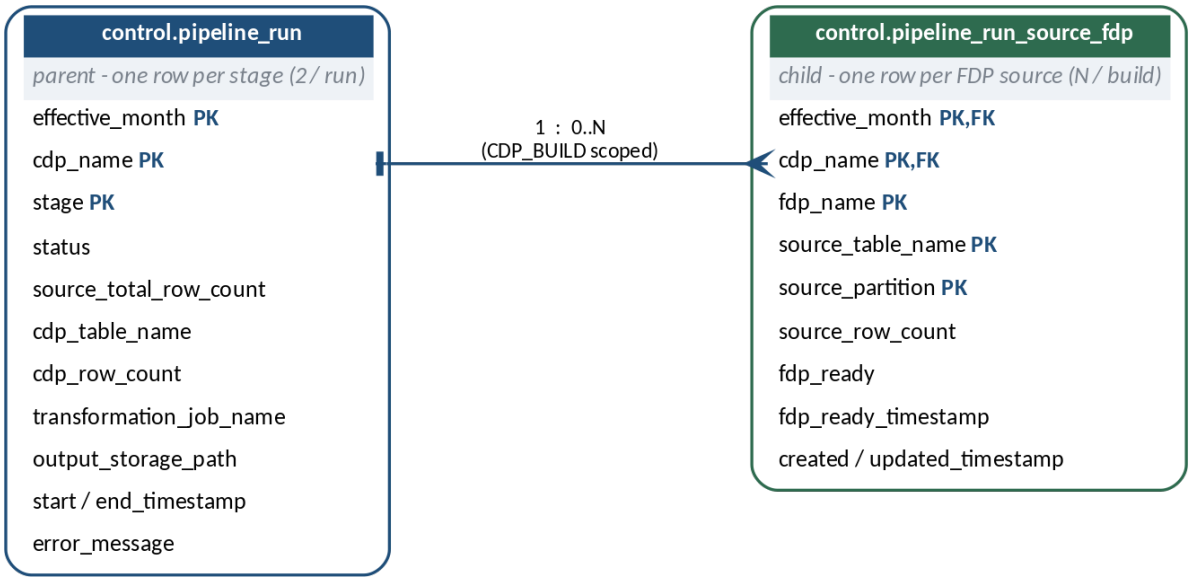
5.1 Model: a config catalogue + parent/child control tables

The metadata is modelled as **three normalised tables** — one **config** catalogue plus two **runtime** state tables — rather than one wide table with repeated ARRAY<STRING> columns:

1. **`control.cdp_source_reference` (catalogue — config).** One row per FDP table/view a CDP consumes. Design-time configuration that defines *what* each CDP is built from; the runtime tables and readiness checks reference it. Presented first, in Section 5.2.
2. **`control.pipeline_run` (parent — stage state). One row per stage of a run.** Every CDP has a `CDP_BUILD` row; it has a `TRANSFORMATION` row **only if it is configured for transformation** (`cdp_registry.has_transformation`, Section 16.1). **Transformation is optional** — the same architecture builds any CDP, and build-only CDPs are consumed directly from the Data Mart (one `CDP_BUILD` stage row only). The orchestration state machine the scheduler reads and writes (Section 5.3).
3. **`control.pipeline_run_source_fdp` (child — runtime lineage & readiness).** One row per contributing FDP source for a run, with row counts and the readiness flag; records *what was actually read* and is validated against the catalogue (Section 5.4). It replaces the old `source_fdp_names` / `source_table_names` / `source_partitions` arrays with proper rows.

The catalogue is presented first because it is the static input the runtime tables build on.

Why normalise the FDP detail out. The arrays coupled three parallel lists by position (fragile), could not carry per-source attributes (row count, readiness flag, ready-timestamp), and made "are all FDPs ready?" an array-unnest query. A child table makes each source a first-class row: per-FDP readiness becomes a simple `COUNT`, per-source row counts roll up to the parent's reconciliation total, and lineage is queryable and indexable. FDP detail is **build-scoped**, so the child relates to the run (not to the `TRANSFORMATION` row).



Grain & keys. Parent grain = one row per `(effective_month, cdp_name, stage)` → exactly two rows per monthly run. Child grain = one row per `(effective_month, cdp_name, fdp_name, source_table_name, source_partition)`. The child foreign-keys to the parent on `(effective_month, cdp_name)` (logically to the `CDP_BUILD` stage row).

Store: transactional, not BigQuery. The control plane lives in a **transactional relational store (Cloud SQL for PostgreSQL; Cloud Spanner at scale)** — *not* in BigQuery. The control plane needs single-row ACID updates, enforced `PRIMARY KEY` / `FOREIGN KEY` / `CHECK` constraints, and row-level locking (`SELECT … FOR UPDATE`) so that the stage-claim is atomic and two scheduler ticks can never double-start a stage. BigQuery is an analytical, append-oriented warehouse: no enforced keys, high-latency small DML, and no row locks — wrong tool for an orchestration state machine. (BigQuery remains the home of the **Data Mart** and the **processing dataset** — data, not control.) The DDL below is PostgreSQL dialect.

5.2 Source reference (catalogue) table — `control.cdp_source_reference`

This is the **config** (design-time) catalogue, presented first because the runtime control tables that follow (Sections 5.3–5.4) reference and seed from it. It catalogues the FDP **tables and views** each CDP consumes. It exists so the **orchestration never hardcodes** the source list — the scheduler and readiness check read it to know which FDP objects to expect for a CDP. The dbt build is necessarily **CDP-specific** (each model references its FDP tables/views directly in SQL/`sources.yml`), so this table **catalogues** those sources and is kept in sync with dbt; it does not generate or replace the dbt code. Its value is a single, queryable record of what each CDP depends on — used for readiness, lineage, and support routing.

```
CREATE TABLE control.cdp_source_reference (
  cdp_name          TEXT    NOT NULL,   -- which Data Mart consumes the source
  fdp_name          TEXT    NOT NULL,   -- owning FDP (and support routing key)
  source_object     TEXT    NOT NULL,   -- fully-qualified FDP table or view, e.g. fdp.party_v2
  object_type       TEXT    NOT NULL,   -- 'TABLE' | 'VIEW'
  partition_pattern TEXT,               -- how the monthly slice is resolved (e.g. monthly DATE)
  required          BOOLEAN NOT NULL DEFAULT TRUE,  -- must be ready before CDP_BUILD starts
  active            BOOLEAN NOT NULL DEFAULT TRUE,  -- soft-enable/disable without deleting
  mapping_sheet_version TEXT,           -- mapping sheet this reference aligns to
  notes             TEXT,
  created_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT pk_cdp_source PRIMARY KEY (cdp_name, fdp_name, source_object),
  CONSTRAINT ck_objtype CHECK (object_type IN ('TABLE','VIEW'))
);
```

What this table gives us. A single, queryable record of **which FDP tables and views each CDP depends on**. A CDP usually draws from several FDPs (and several objects within them), so a CDP has **multiple rows** — one per source object. From it we can answer, with one query: what does this CDP consume; which CDPs would a given FDP change affect; what must be ready before a build; and who owns each source.

Grain — one row per source object (a CDP has many rows). Example: `cdp_customer_360` drawing from four FDPs, including two views:

cdp_name	fdp_name	source_object	object_type	required	active
cdp_customer_360	fdp_party	fdp.party_v2	TABLE	TRUE	TRUE
cdp_customer_360	fdp_party	fdp.party_address_vw	VIEW	TRUE	TRUE
cdp_customer_360	fdp_account	fdp.account_v3	TABLE	TRUE	TRUE
cdp_customer_360	fdp_txn	fdp.txn_monthly	TABLE	TRUE	TRUE
cdp_customer_360	fdp_ml	fdp.churn_score_vw	VIEW	FALSE	TRUE

How it is used:

- **Readiness & seeding (orchestration — no hardcoding).** The driver reads the `active, required` rows for a `cdp_name` and seeds the expected `pipeline_run_source_fdp` child rows; the readiness gate then checks every expected source is ready — the expected set comes from this table, not a hardcoded list in the DAG. (`required` = FALSE rows, e.g. an optional/pending view, are catalogued but do not block the build.)
- **Catalogue of dbt's sources.** The dbt build is CDP-specific and references its FDP tables/views directly; this table **mirrors** that list (ideally generated/validated from dbt `sources.yml` in CI) so the catalogue stays truthful — it records what dbt uses, it does not drive dbt.
- **Lineage & impact analysis.** "Which CDPs/CSVs depend on `fdp.party_v2`?" is a single `WHERE source_object = …` query — essential when an FDP team changes a table or view.
- **Support routing.** `fdp_name` is the key that routes an FDP problem to its owning team (see the run book support model).

It is the **object-level companion** to the field-level mapping sheet (Section 13.1): the mapping sheet says *which fields*, this table says *which tables/views* — and the runtime child table (Section 5.4) records *what was actually read* each month.

5.3 Parent DDL — `control.pipeline_run` (Cloud SQL / PostgreSQL)

```
CREATE TABLE control.pipeline_run (
  effective_month         DATE        NOT NULL,   -- run month (1st of month)
  cdp_name                TEXT        NOT NULL,   -- Data Mart identifier
  stage                   TEXT        NOT NULL,   -- 'CDP_BUILD' | 'TRANSFORMATION'
  status                  TEXT        NOT NULL DEFAULT 'NOT_STARTED',
  source_total_row_count  BIGINT,                 -- SUM(child.source_row_count); CDP_BUILD only
  cdp_table_name          TEXT,                   -- built mart table (CDP_BUILD)
  cdp_row_count           BIGINT,                 -- mart output rows (CDP_BUILD)
  start_timestamp         TIMESTAMPTZ,
  end_timestamp           TIMESTAMPTZ,
  transformation_job_name TEXT,                   -- Dataflow job name/id (TRANSFORMATION)
  output_storage_path     TEXT,                   -- gs:// path of final CSV (TRANSFORMATION)
  error_message           TEXT,
  created_timestamp       TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_timestamp       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT pk_pipeline_run PRIMARY KEY (effective_month, cdp_name, stage),
  CONSTRAINT ck_stage  CHECK (stage  IN ('CDP_BUILD','TRANSFORMATION')),
  CONSTRAINT ck_status CHECK (status IN ('NOT_STARTED','STARTED','COMPLETED','ERROR'))
);
CREATE INDEX ix_run_lookup ON control.pipeline_run (cdp_name, effective_month, stage, status);
```

5.4 Child DDL — `control.pipeline_run_source_fdp` (Cloud SQL / PostgreSQL)

```
CREATE TABLE control.pipeline_run_source_fdp (
  effective_month     DATE        NOT NULL,   -- part of FK -> pipeline_run
  cdp_name            TEXT        NOT NULL,   -- part of FK -> pipeline_run
  fdp_name            TEXT        NOT NULL,   -- contributing FDP (contract input)
  source_table_name   TEXT        NOT NULL,   -- physical FDP table consumed
  source_partition    TEXT        NOT NULL,   -- partition / snapshot read (e.g. '2026-06')
  source_row_count    BIGINT,                 -- rows read from this table/partition
  fdp_ready           BOOLEAN     NOT NULL DEFAULT FALSE,
  fdp_ready_timestamp TIMESTAMPTZ,            -- when readiness was signalled
  created_timestamp   TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_timestamp   TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT pk_run_source PRIMARY KEY
    (effective_month, cdp_name, fdp_name, source_table_name, source_partition),
  CONSTRAINT fk_run FOREIGN KEY (effective_month, cdp_name, stage_ref)
    -- FK targets the CDP_BUILD stage row; see note below
    REFERENCES control.pipeline_run (effective_month, cdp_name, stage)
);
```

> **FK note.** Because the parent PK includes `stage`, the child references the build row. Two clean options: (a) add a generated/constant column `stage_ref = 'CDP_BUILD'` on the child and FK on `(effective_month, cdp_name, stage_ref)` as above; or (b) FK to a thinner `pipeline_run_header (effective_month, cdp_name)` parent and keep `stage` only on a `pipeline_run_stage` table. Option (a) is simplest; option (b) is the most normalised. Pick one at build time.

5.5 Field-by-field data dictionary

Parent — `control.pipeline_run` (“Stage” = which stage row the field is meaningful for):

#	Field	Type	Key / Null	Populated by (when)	Stage	Description
1	`effective_month`	`DATE`	PK / No	Scheduler @ insert	Both	Logical month processed; normalised to the 1st. Partition key.
2	`cdp_name`	`STRING`	PK / No	Scheduler @ insert	Both	Target Data Mart id (e.g. cdp_customer_360). Cluster key.
3	`stage`	`STRING`	PK / No	Scheduler @ insert	Both	Stage this row tracks: CDP_BUILD or TRANSFORMATION. Cluster key.
4	`status`	`STRING`	— / No	Owning job @ each transition	Both	Lifecycle: NOT_STARTED → STARTED → COMPLETED / ERROR. Drives gating.

#	Field	Type	Key / Null	Populated by (when)	Stage	Description
5	source_total_row_count	INT64	— / Yes	DBT (build)	BUILD	SUM(child.source_row_count); top of the reconciliation chain.
6	cdp_table_name	STRING	— / Yes	DBT @ complete	BUILD	Fully-qualified mart table produced by the build.
7	cdp_row_count	INT64	— / Yes	DBT @ complete	BUILD	Row count of built mart; reconciled vs inputs and CSV.
8	start_timestamp	TIMESTAMP	— / Yes	Owning job @ STARTED	Both	When the stage began; basis for duration / SLA.
9	end_timestamp	TIMESTAMP	— / Yes	Owning job @ terminal	Both	When the stage reached COMPLETED / ERROR.
10	transformation_job_name	STRING	— / Yes	Scheduler / Dataflow @ launch	TRANSFORMATION	Dataflow job name / id; links to the Dataflow console.
11	output_storage_path	STRING	— / Yes	Dataflow @ complete	TRANSFORMATION	GCS path of the final CSV; durable hand-off pointer.
12	error_message	STRING	— / Yes	Owning job @ failure	Both	Captured failure reason when status = ERROR.
13	created_timestamp	TIMESTAMP	— / No	Scheduler @ insert	Both	Immutable row-creation audit timestamp.
14	updated_timestamp	TIMESTAMP	— / No	Any writer @ every update	Both	Last-modified audit timestamp; bumped on every transition.

Child — `control.pipeline_run_source_fdp` (all rows relate to the `CDP_BUILD` stage):

#	Field	Type	Key / Null	Populated by (when)	Description
1	effective_month	DATE	PK, FK / No	Scheduler / FDP @ insert	Run month; FK to parent. Partition key.
2	cdp_name	STRING	PK, FK / No	Scheduler / FDP @ insert	Target Data Mart; FK to parent. Cluster key.
3	fdp_name	STRING	PK / No	FDP publisher	Contributing curated FDP (contract input). Cluster key.
4	source_table_name	STRING	PK / No	FDP publisher	Physical FDP table consumed by the build.
5	source_partition	STRING	PK / No	FDP publisher	Exact partition / snapshot read (e.g. 2026-06).
6	source_row_count	INT64	— / Yes	FDP / DBT	Rows read from this table/partition; rolls up to parent total.
7	fdp_ready	BOOL	— / No	FDP publisher @ readiness	Whether this source has published & self-validated.
8	fdp_ready_timestamp	TIMESTAMP	— / Yes	FDP publisher @ readiness	When readiness was signalled; basis for FDP SLA.
9	created_timestamp	TIMESTAMP	— / No	Writer @ insert	Row-creation audit timestamp.
10	updated_timestamp	TIMESTAMP	— / No	Writer @ update	Last-modified audit timestamp.

Population timeline. FDP publishers **insert/update child rows** with their source detail and flip `fdp_ready = TRUE` when validated. The scheduler **inserts the parent stage rows** (`status = NOT_STARTED`); on claim it sets STARTED + `start_timestamp`. DBT rolls the child `source_row_count` into the parent `source_total_row_count`, writes `cdp_table_name` / `cdp_row_count`, then sets `COMPLETED` — or ERROR + `error_message`. `updated_timestamp` is rewritten on every write.

Reconciliation chain. `SUM(child.source_row_count)` → parent `source_total_row_count` → `cdp_row_count` → CSV row count are compared at each boundary; a breach beyond tolerance alerts (risk R9).

Example rows (one run, `cdp_customer_360`, June 2026).

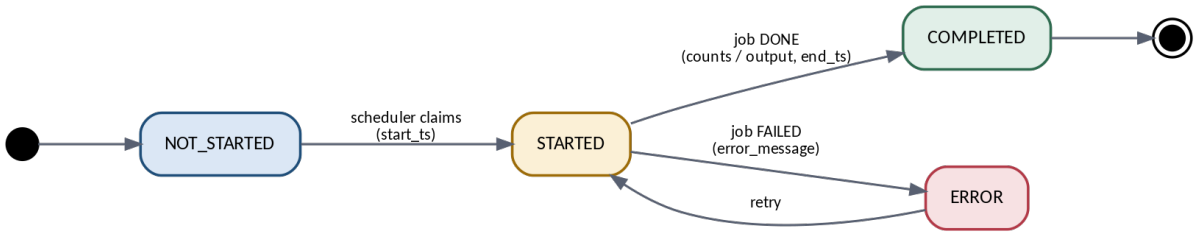
Parent — `pipeline_run` (exactly two rows; shown as field → value per stage):

Field	CDP_BUILD row	TRANSFORMATION row
effective_month	2026-06-01	2026-06-01
cdp_name	cdp_customer_360	cdp_customer_360
stage	CDP_BUILD	TRANSFORMATION
status	COMPLETED	COMPLETED
source_total_row_count	5,140,220	—
cdp_table_name	mart.cdp_customer_360_202606	—
cdp_row_count	4,812,004	—
transformation_job_name	—	df-cdp_customer_360-202606
output_storage_path	—	gs://out/cdp_customer_360/2026-06/

Child — `pipeline_run_source_fdp` (one row per FDP source; shown as field → value per source):

Field	source 1	source 2	source 3
effective_month	2026-06-01	2026-06-01	2026-06-01
cdp_name	cdp_customer_360	cdp_customer_360	cdp_customer_360
fdp_name	fdp_party	fdp_account	fdp_txn
source_table_name	fdp.party_v2	fdp.account_v3	fdp.txn_monthly
source_partition	2026-06	2026-06	2026-06
source_row_count	1,204,880	2,310,540	1,624,800
fdp_ready	TRUE	TRUE	TRUE

5.6 Status state machine



5.7 Control queries

FDP readiness gate (start CDP_BUILD) — completeness + readiness. The gate must validate **all expected sources exist *and* are ready**, measured against the reference table (Section 5.2), not just over whatever rows happen to be present. The check below anti-joins the *expected* set (`cdp_source_reference`) against the *actual* runtime rows (`pipeline_run_source_fdp`) and returns the offenders; **zero rows returned ⇒ start the build.**

```
-- Lists every required source that is MISSING or NOT READY. Empty result = gate open.
SELECT r.fdp_name, r.source_object,
       CASE WHEN a.cdp_name IS NULL THEN 'MISSING' ELSE 'NOT_READY' END AS issue
FROM   control.cdp_source_reference r
LEFT JOIN control.pipeline_run_source_fdp a
       ON a.cdp_name        = r.cdp_name
      AND a.fdp_name        = r.fdp_name
      AND a.source_table_name = r.source_object
      AND a.effective_month = @effective_month
WHERE  r.cdp_name = @cdp_name
  AND  r.active = TRUE
  AND  r.required = TRUE
  AND  (a.cdp_name IS NULL          -- expected source never published a row  (MISSING)
        OR a.fdp_ready = FALSE);    -- published but not validated yet        (NOT_READY)
```

This is what "validate all exist" means: `MISSING` catches a required FDP table/view that produced no row at all (the case a plain `LOGICAL_AND` over present rows would silently pass); `NOT_READY` catches one present but not yet validated. Row-count validation is separate and handled by the reconciliation chain (`SUM(child.source_row_count)` → parent `source_total_row_count` → `cdp_row_count` → CSV; Sections 5.4 and 12h).

Transformation gate (authoritative). Operates purely on the parent stage rows:

```
-- Returns the run iff transformation is allowed to start
SELECT b.cdp_name, b.effective_month
FROM   control.pipeline_run b
JOIN   control.pipeline_run t
  ON   t.cdp_name = b.cdp_name
 AND   t.effective_month = b.effective_month
WHERE  b.stage = 'CDP_BUILD'       AND b.status = 'COMPLETED'
  AND  t.stage = 'TRANSFORMATION'  AND t.status = 'NOT_STARTED'
  AND  b.cdp_name = @cdp_name
  AND  b.effective_month = @effective_month;
```

Atomic stage claim (anti-double-run). Because the store is transactional, the claim is a single atomic statement — no application-side locking needed:

```sql
-- Claims the stage iff still NOT_STARTED; returns 1 row on success, 0 if already claimed
UPDATE control.pipeline_run
   SET status = 'STARTED', start_timestamp = now(), updated_timestamp = now()
 WHERE cdp_name = @cdp_name AND effective_month = @effective_month
   AND stage = @stage AND status = 'NOT_STARTED'
RETURNING cdp_name, effective_month, stage;
```

Two concurrent scheduler ticks race on the same row; the database serialises them and exactly one `UPDATE` matches `status='NOT_STARTED'`. The loser gets zero rows and backs off. This `RETURNING`-guarded update (or `SELECT … FOR UPDATE` then `UPDATE`) is the idempotency / anti-double-run guard — and is precisely what a transactional store gives us that BigQuery cannot.

6. Detailed Component Responsibilities

6.1 FDP Layer — the contract boundary

FDPs are curated, versioned, quality-checked data products owned by upstream teams. They are the **only** input the mart may read; the mart never touches raw source systems. Each FDP exposes a **contract**: a stable schema, defined partitioning, freshness/SLA guarantees, and a readiness signal. The FDP's responsibilities here are to (a) produce the month's data, (b) self-validate against its contract, and (c) publish a readiness row per source (`fdp_name`, `source_table_name`, `source_partition`, `source_row_count`, `fdp_ready`) into the child `pipeline_run_source_fdp` table. This decouples mart development from source volatility: as long as the contract holds, upstream internals can change freely.

6.2 Metadata Table — the control plane

The single source of truth for *what is allowed to run and what has run*. It carries no business data — only orchestration state, lineage, counts, timing, and errors, split across the parent `pipeline_run` (stage state) and child `pipeline_run_source_fdp` (input lineage/readiness). It is hosted in a **transactional store (Cloud SQL/Spanner), not BigQuery**, because it needs ACID single-row updates, enforced keys, and row locking for the atomic stage-claim. Every stage transition is recorded here, which makes the pipeline auditable (who ran what, when, with which inputs and outputs) and restartable (state survives any individual job dying). It is the integration contract **between** the scheduler and the compute stages: jobs never talk to each other, they only read/write these tables.

6.3 Scheduler (TWS) — orchestration

The only orchestrator. It owns the calendar (monthly trigger), launches Dockerised DBT and the Dataflow job, enforces single-run/no-parallelism, and evaluates gates by reading metadata. Critically it performs **no business logic** and holds **no direct job-to-job links** — its dependencies are metadata predicates. This keeps orchestration declarative and lets operators reason about the run purely from the metadata table.

6.4 DBT Execution Layer (Docker)

DBT builds the Data Mart from FDP inputs: models (SQL transformations), tests (contract + data-quality assertions), and snapshots where history is needed. Running in Docker pins the dbt version, adapters, and dependencies for reproducibility across environments and over time. On success it writes `cdp_table_name`, `cdp_row_count`, and `COMPLETED`; on model or test failure it writes `ERROR` with a message and halts the run (no transformation downstream).

6.5 Dataflow Processing Layer (Java / Apache Beam)

A parameterised Beam pipeline that reads the built mart, applies the mapping template, stages intermediate results in the processing dataset, and writes the final CSV. It is a managed, autoscaling, parallel compute layer — appropriate for transformations heavier or more procedural than is comfortable in SQL/DBT. See Section 7 for detail.

6.6 Processing Dataset (BigQuery) — intermediate working store

A dedicated BigQuery dataset for the Dataflow job's intermediate/working tables: normalised inputs, join/aggregation intermediates, and pre-CSV shaping. Separating it from the curated mart keeps the mart clean and read-only to the transform, gives the job a scratch area it fully owns, and isolates transform retries from mart data. Tables here are transient and should carry the `effective_month`/`cdp_name` in their names and a TTL/expiration for hygiene.

6.7 Cloud Storage — final output

The delivery surface for downstream consumers: final CSV file(s) written to a deterministic `output_storage_path` (partitioned by `cdp_name`/`effective_month`). Storage is the durable, decoupled hand-off boundary — consumers pull from a path, they never reach back into BigQuery or the mart. A manifest/_SUCCESS marker and the metadata `output_storage_path` make completion unambiguous.

6.8 Data Mart & processing dataset — partitioning and clustering

These are **BigQuery** tables (only the *control store* is transactional), so partitioning and clustering are the primary cost/performance levers. The guiding principle: **partition on the column the monthly run filters on (`effective_month`) so the Dataflow read prunes to a single partition, and cluster on the entity keys used for filtering and joining.**

Data Mart (CDP) tables. Two physical patterns, pick per mart:

- **History table (recommended):** one table holding all months, **partitioned by `effective_month` at MONTH granularity**, with `require_partition_filter = TRUE` so no query can full-scan across months. **Cluster by the 2–4 business keys** the transform and consumers filter/join on (highest-value filter first), e.g. `entity_id`, `segment`, `region`. The Dataflow read for one `effective_month` then touches exactly one partition, and entity-level predicates/joins hit only the relevant clusters.
- **Per-month table (`cdp_..._YYYYMM`):** if you prefer one immutable table per run, the month *is* the table, so partitioning adds little — still **cluster by the entity keys**, and optionally partition by a business event/as-of date if the data spans days within the month.

```sql
-- History-table pattern for a Data Mart
CREATE TABLE mart.cdp_customer_360 (
  effective_month DATE NOT NULL,     -- 1st of month; the run/partition key
  customer_id     STRING,
  segment         STRING,
  region          STRING,
  -- ... mart columns ...
)
PARTITION BY DATE_TRUNC(effective_month, MONTH)
CLUSTER BY customer_id, segment, region          -- <= 4 cols, most-filtered first
OPTIONS (require_partition_filter = TRUE);
```

Processing dataset (intermediate) tables. Partition by `effective_month` as well so each run's working data is isolated and prunable; **cluster by the join keys** used between the read and aggregation stages (e.g. `customer_id`). Always set **table/partition expiration (TTL)** since these are transient (e.g. `partition_expiration_days`), and namespace by `cdp_name+effective_month` (Section 7.4).

Rules of thumb / cautions.

- Choose the partition column = the run filter (`effective_month`); align it with the Dataflow read predicate so pruning actually happens.
- Cluster by columns that appear in `WHERE`/`JOIN`/`GROUP BY`, ordered most-selective-and-frequently-filtered first; clustering beyond ~4 columns yields little.
- Don't partition on a very high-cardinality column (BigQuery caps at ~10k partitions, and one-month-per-partition keeps you well within it); don't cluster on `effective_month` (it's already the partition).
- Set `require_partition_filter` on the mart to make accidental full scans fail fast — a cost guardrail for a wide, growing table.

---

7. Dataflow Design (Java)

7.1 How the job works

The job is an Apache Beam pipeline packaged as a Java application and executed on the Cloud Dataflow runner. Logical pipeline:

It reads only the relevant `effective_month` slice of `cdp_name` (predicate/partition pushdown), transforms in parallel across workers, materialises intermediates to the processing dataset for the heavier join/aggregate phase, then formats and writes CSV with a deterministic naming/sharding scheme plus a success marker.

7.2 How parameters are passed

Three parameters are passed at submit time via Beam **pipeline options** (`ValueProvider`/custom `PipelineOptions`), supplied by the scheduler's launch job:

Parameter	Purpose	Example
`cdp_name`	Which mart to read; selects source dataset/table	`cdp_customer_360`
`effective_month`	Which monthly slice to process; drives read predicate, intermediate/CSV naming, idempotency	`2026-06-01`
`mapping_template`	Which transformation spec to apply (column mapping, derivations, output schema)	`tmpl_v3_regulatory`

Because parameters are explicit at launch, the same artifact handles every mart/month without code change — the scheduler simply varies the three values. The job echoes `transformation_job_name` and resolved `output_storage_path` back to metadata.

7.3 How mapping templates are applied

The `mapping_template` is a versioned, externalised specification (e.g. JSON/YAML in Cloud Storage or a config table) that declares source→target field mappings, derivations/expressions, type casts, filters, and the output CSV schema/order. At startup the job loads and validates the template, then drives a generic `MapElements`/`DoFn` from it rather than hard-coding columns. This makes output changes a **config** change, supports multiple marts with one codebase, and version-stamps every run (the template version belongs in lineage/audit). Templates should be validated against the mart schema in `TRANSFORMATION_GATE`/job startup so a bad template fails fast, not mid-write.

7.4 How the processing dataset is used

The job uses the processing dataset as its working store: write normalised/intermediate tables, then read them back for the join/aggregation stage that is awkward to do purely in-stream. Conventions: namespace tables by `cdp_name+effective_month` (e.g. `proc_cdp_customer_360_202606_step1`), set table expiration/TTL, and **truncate-or-replace per run** so a retry starts clean. This isolates transform scratch from the curated mart and from other runs, and makes intermediate results inspectable for debugging.

7.5 Chunking & in-job parallelism

The transform never loads a CDP in one piece; data is processed in chunks at three levels, which is what lets a single job scale and stay within memory.

- **Read is pre-chunked by partition, then split.** The job reads only the `effective_month` partition (the coarse chunk — see Section 6.8), and the **BigQuery Storage Read API** splits that partition into multiple parallel **streams**. Beam distributes streams across workers, so the read is chunked and parallel from the start.
- **Processing in bundles.** Beam groups elements into **bundles** and runs the mapping `DoFn` on them across workers; Dataflow **autoscaling** adds/removes workers with the backlog, and dynamic work rebalancing re-splits hot chunks. Mapping is stateless and per-row, so it parallelises cleanly.

- **Aggregations chunk by key.** Any join/aggregate is keyed (e.g. by `customer_id`), so work is partitioned by key rather than held in one place; use `Combine`/partial aggregation to avoid materialising a whole group in memory. Heavy joins are pushed down to the processing dataset (Section 7.4) and read back in splits rather than shuffled entirely in-stream.
- **Output is chunked into shards.** The CSV is written as **N shard files** (`part-00000-of-000NN.csv`); shard count is tuned to a target file size so no single file is unwieldy, and a `_SUCCESS` marker plus `manifest.json` (shard list + row count) makes the chunk set verifiable. Downstream CDS pre-processing reads the shard set, not one giant file. (Full knobs in Section 14.3.)
- **For an exceptionally large CDP.** If one CDP outgrows comfortable single-job processing, chunk it explicitly by a sub-key range (e.g. by `region/segment` or a hash bucket) and process the chunks — either as parallel `DoFn` branches or, if needed, separate sub-jobs whose shards land in the same output prefix. This is the in-CDP analogue of the cross-CDP fan-out and is the first lever before splitting the job by domain (Section 8).

Net: partition pruning gives the coarse chunk, the Storage API + Beam bundles give automatic fine-grained chunking and parallelism, key-based aggregation bounds memory, and output sharding chunks the result — all without manual batching in the common case.

---

8. Key Design Decisions

Each decision below states its rationale and the trade-off it carries; none is without cost.

Metadata-driven orchestration. Direct job chaining — where the build job invokes the transform — couples jobs to one another and hides run state inside the scheduler. Centralising all state in a single queryable control plane provides four benefits: an audit trail and lineage; restartability, since a failed job leaves a known state from which the run resumes; gating that the scheduler and operators evaluate identically; and a clean boundary between orchestration and compute. The trade-off is that the control store becomes Tier-1 infrastructure and the gate checks add minor latency — both acceptable for a monthly batch.

A transactional control store, not BigQuery. This is a settled decision rather than an open risk. The control plane requires single-row ACID updates, enforced PK/FK/CHECK constraints, and row-level locking so that the stage claim is atomic — capabilities a transactional store (Cloud SQL for PostgreSQL; Spanner at scale) provides and BigQuery does not. BigQuery remains the home of the Data Mart and processing dataset (data, not control). Because the design deliberately places a live database in the execution path, it is engineered accordingly: managed HA (regional Cloud SQL / multi-region Spanner), point-in-time-recovery backups, and connection pooling from the jobs. The workload is monthly and low-concurrency, so the instance stays small. Store integrity and availability are handled here by enforced constraints, managed HA, and PITR — they are not carried as risks. The only residual is an orphaned run left by a crashed worker, which the database cannot detect; that is R1.

A fixed CDP schema, filled incrementally. The complete set of fields a CDP must expose is known up front and captured in the **mapping sheet** (the FDP-field → CDP-field specification). The CDP target schema is therefore fixed from the outset and built in full, even before the FDPs surface every source field. Fields without an available source are materialised as typed `NULL` placeholders; as FDPs add fields, the mapping sheet and dbt model are updated to populate them. The CDP's shape never changes — only the completeness of its data. This is the incremental build: a stable contract with growing coverage (detail in Section 13.1). The benefit is significant: downstream consumers and the Dataflow transform bind to a stable schema immediately and are never exposed to a breaking change, and a useful CDP can ship early rather than waiting on every FDP field. The cost is that some columns remain `NULL` for a period and the mapping sheet must be maintained as a governed artifact.

FDPs as the only input. Routing all input through curated FDPs, with no direct source access, makes the mart depend on **stable contracts rather than volatile source systems**. Upstream teams can re-platform their internals freely; provided the contract holds, the mart is unaffected. Data-quality and lineage are also consolidated into one reusable layer rather than re-implemented by each mart. The cost is an additional layer of indirection and a dependency on FDP teams meeting their readiness window — tracked explicitly (see R5).

Separation of build and transform. `CDP_BUILD` is set-based SQL in DBT; `TRANSFORMATION` is procedural Java/Beam that shapes a CSV. Combining them into a single tool would compromise both, so each stage uses the technology suited to it and fails, retries, and scales independently. The mart is also a reusable asset that may serve many consumers, so the transform can be re-run against an already-built mart without rebuilding it, and either stage can evolve without affecting the other. The cost is one additional hop and the gate latency between stages.

One Dataflow job per CDP. Each CDP runs its own parameterised Dataflow job (`df-<cdp>-<month>`), and these jobs run in parallel across CDPs; only within a CDP is the transform single. This keeps each job simple to version and reason

about while still scaling horizontally by CDP. The trade-offs: per CDP, that single job remains a blast-radius unit — a defect or a bad month affects that CDP's entire transform — and its template logic can grow large; and many concurrent jobs place pressure on shared GCP quotas, managed through the driver's `max_concurrency` and a dedicated BigQuery reservation (see R4). Should a single CDP ever outgrow one job, it can be split by domain; cross-CDP throughput is already parallel by design.

---

9. Risks & Mitigations

The principal risks are listed below in approximate priority order, each with its mitigation. **"Inherent" is the raw impact / likelihood; "Residual" is the rating once the mitigation is in place** — these are managed, residual risks for this first version (v1), to be re-rated as the build proceeds.

#	Risk	Inherent (I / L)	Mitigation	Residual
R1	**Orphaned / stuck run state** — a build or transform process dies *after* claiming `STARTED`, leaving the stage stuck so the run cannot advance. (The transactional store already prevents corruption, double-claims, and availability loss by design — see Section 8 — so those are not risks; a crashed worker is the residual it cannot detect.)	Med / Low	Heartbeat + stuck-`STARTED` detector (alert when `STARTED` exceeds expected duration); operator/auto reset to `NOT_STARTED` per runbook (Section 19); safe idempotent re-run via the atomic claim	**Low**
R2	**Processing dataset contention / collisions** — with **all CDPs running in parallel**, intermediates could clash or a retry could clobber a live run	Med / Low	Per-CDP+month table namespacing (`proc.<cdp>_<YYYYMM>_<step>`) makes parallel CDPs collision-free by construction; `WRITE_TRUNCATE` per run; table TTL; atomic single-run-per-CDP claim; per-CDP dataset if extra isolation wanted; driver `max_concurrency` caps load	**Low**
R3	**Single pipeline service account** — one account spans FDP read, mart/proc write, GCS write, and control-DB access, so a compromise has broad blast radius. Accepted as a deliberate trade-off: all marts feed one CDS pre-processing, so the pipeline is a single trust domain.	High / Low	Roles scoped to specific datasets/buckets (not project-wide); Workload Identity (no keys); VPC-SC perimeter; rotation + audit logging; deny-by-default IAM	**Low**
R4	**Shared-quota saturation under parallelism** — many CDPs at once exhaust BigQuery slots, Dataflow CPU/IP quota, or Cloud SQL connections	Med / Med	Driver `max_concurrency` cap; dedicated BigQuery reservation for the build window; raise Dataflow regional quota + connection pooling to Cloud SQL; Beam autoscaling + partition pruning; load-test the peak (widest) month	**Med**

#	Risk	Inherent (I / L)	Mitigation	Residual
R5	**FDP not ready / late** — build cannot start, monthly SLA missed	Med / Med	Per-source readiness in the child `pipeline_run_source_fdp` table (`fdp_ready`, `fdp_ready_timestamp`); `LOGICAL_AND(fdp_ready)` gate + timeout alert (`CDP_BUILD_ALERT_NOT_READY`); per-FDP SLA tracking; partial-readiness reporting; clear upstream contract + escalation	**Med**
R6	**DBT build or test failure** — mart wrong or absent	High / Med	`dbt test` contracts + DQ as a hard gate; `ERROR` halts before transform; row-count reconciliation (`source_total_row_count` vs `cdp_row_count`); env pinned via Docker image digest	**Low**
R7	**Partial / duplicate CSV output** — consumers read incomplete data	High / Low	Write to temp path then atomic rename; `_SUCCESS`/manifest marker; `TRANSFORMATION_OUTPUT_VERIFY` reconciles counts; consumers gate on metadata `COMPLETED` + marker, not file presence	**Low**
R8	**Idempotency / double trigger** — two scheduler ticks start the same stage	Med / Low	Authoritative gate (`BUILD=COMPLETED AND TRANSFORMATION=NOT_STARTED`) + **atomic transactional claim** (`UPDATE … WHERE status='NOT_STARTED' RETURNING`); DB-enforced unique PK `(effective_month,cdp_name,stage)`; jobs re-assert gate at startup	**Low**
R9	**Silent data skew vs source** — output silently wrong	Med / Med	Reconcile per-source `SUM(child.source_row_count)` → parent `source_total_row_count` → `cdp_row_count` → CSV row count at each boundary; alert on threshold breach	**Low**
R10	**BigQuery mart cost / full-table scans** — a missing partition filter on a growing history mart scans every month, inflating cost and runtime	Med / Med	Partition mart + processing tables by `effective_month`; `require_partition_filter = TRUE` on the mart; cluster by entity keys; align Dataflow read predicate to the partition (Section 6.8); cost/bytes-scanned monitoring	**Low**

10. Operational Notes (for build kickoff)

- **Idempotency contract:** every job must be safe to re-run; the metadata claim + per-run intermediate naming are the mechanisms. Re-running a `COMPLETED` stage must be a no-op.
- **Restart semantics:** on `ERROR`, operators reset the stage row to `STARTED`/`NOT_STARTED` per runbook; the scheduler resumes from metadata, not from a job pointer.
- **Observability:** alert on `ERROR`, on `STARTED` exceeding an expected-duration SLA, and on reconciliation breaches. Surface the metadata table as the single operational dashboard.
- **Security posture:** single least-privilege pipeline SA with dataset/bucket-scoped roles (R3); VPC-SC around BigQuery/GCS/Cloud SQL; no long-lived keys (Workload Identity).
- **Environments:** Docker image and Dataflow template are promoted by digest/version through dev→test→prod; `mapping_template` versions are tracked in lineage.

10.1 Decisions & open questions for the review

1. **Decided — control store is transactional (Cloud SQL for PostgreSQL; Spanner at scale), not BigQuery.** Confirm which managed engine and the HA/region topology. Open sub-question: child-FK shape — generated `stage_ref` column (Section 5.4 option a) vs split header/stage tables (option b).
2. **Decided — all CDPs run in parallel** (Section 4.4); only a single CDP's own run is serialized. Confirm the driver's `max_concurrency` value and the BigQuery reservation / Dataflow quota headroom for the widest month.
3. Retention/TTL policy for processing-dataset intermediates and historical CSV outputs.
4. Where `mapping_template` versions live and how they are governed/approved.
5. Desired monthly **SLA** and the FDP-readiness cutoff time, plus the **volumetrics** (Section 22.1) and the **SLO targets** (Section 23) — all currently *TBD*.

Part B — Build Specification

This part is the engineering build-out, written concretely so a team can begin implementation directly. Where a choice remains open, a **default** is stated and flagged; defaults are decisions to confirm, not hard constraints. Assumed stack: **Cloud SQL for PostgreSQL** (control), **BigQuery** (mart + processing), **dbt-bigquery in Docker** (build), **Apache Beam Java on Cloud Dataflow Flex Templates** (transform), **GCS** (output), **Tivoli Workload Scheduler** (orchestration).

11. Naming Conventions & Environments

Stable names are foundational: the scheduler, SQL, and jobs all derive their paths from them, so these should be fixed first.

Thing	Convention	Example
`effective_month`	`YYYY-MM-01` (first of month)	`2026-06-01`
Control DB	Cloud SQL Postgres, schema `control`	`control.pipeline_run`, `control.pipeline_run_source_fdp`
FDP datasets (read-only)	`fdp_*`	`fdp.party_v2`
Mart dataset / table	`mart.cdp_<name>` (history, partitioned)	`mart.cdp_customer_360`
Processing dataset / table	`proc.<cdp>_<YYYYMM>_<step>`	`proc.cdp_customer_360_202606_step1`
Output bucket / prefix	`gs://<env>-cdp-output/<cdp_name>/<YYYY-MM>/`	`gs://prod-cdp-output/cdp_customer_360/2026-06/`
Output files	`part-*.csv`, plus `_SUCCESS` and `manifest.json`	`part-00000-of-00003.csv`
Dataflow job name	`df-<cdp_name>-<YYYYMM>`	`df-cdp_customer_360-202606`
TWS jobs	`J_<VERB>_<OBJECT>`	`CDP_BUILD_DBT_RUN`
Service account	`sa-cds-pipeline@<project>.iam`	one pipeline SA, all stages (Section 17)
Environments	`dev` → `test` → `prod`, isolated projects	image/template promoted by digest

Parameters that flow through every layer: `cdp_name`, `effective_month`, and (transform only) `mapping_template`. Everything else (table names, paths, job names) is **derived** from these three by the conventions above — engineers should never hand-type a path.

GCP service mapping (this is a Google Cloud build — concrete services per component):

Component	GCP service	Notes
Control store	**Cloud SQL for PostgreSQL** (regional HA)	ACID claim, PK/FK; Spanner if multi-region scale needed
Data Mart + processing	**BigQuery**	mart + proc datasets; dedicated **reservation** for build window
Mart build	**dbt-bigquery in a container** on Cloud Run Jobs / GKE	image in **Artifact Registry**
Transform	**Dataflow** (Apache Beam Java, **Flex Templates**)	one job per CDP, autoscaling
Final output	**Cloud Storage**	per-CDP/month prefix + _SUCCESS
Orchestration	**Tivoli Workload Scheduler** (on GCE/GKE)	the only orchestrator; calls `gcloud`/SDK
Images / templates	**Artifact Registry** + GCS template specs	promoted by digest dev→test→prod
Identity	**IAM + Workload Identity**	single pipeline SA, no keys
Network/data boundary	**VPC Service Controls**	perimeter around BigQuery/GCS/Cloud SQL
Config / mapping template	**Cloud Storage** (`gs://<env>-cdp-config/`)	versioned JSON

12. Metadata Operations (concrete SQL)

All statements run against Cloud SQL Postgres as the pipeline service account `sa-cds-pipeline`. Every write also sets `updated_timestamp = now()` (trigger or explicit).

(a) Seed the run at month open — scheduler creates the stage rows. Always seed `CDP_BUILD`; seed `TRANSFORMATION` **only when the CDP is configured for it** (`cdp_registry.has_transformation`). A build-only CDP gets one row.

```
-- CDP_BUILD: always
INSERT INTO control.pipeline_run (effective_month, cdp_name, stage, status, created_timestamp,
 ↪  updated_timestamp)
VALUES (@effective_month, @cdp_name, 'CDP_BUILD', 'NOT_STARTED', now(), now())
ON CONFLICT (effective_month, cdp_name, stage) DO NOTHING;

-- TRANSFORMATION: only if has_transformation
INSERT INTO control.pipeline_run (effective_month, cdp_name, stage, status, created_timestamp,
 ↪  updated_timestamp)
SELECT @effective_month, @cdp_name, 'TRANSFORMATION', 'NOT_STARTED', now(), now()
FROM   control.cdp_registry r
WHERE  r.cdp_name = @cdp_name AND r.has_transformation
ON CONFLICT (effective_month, cdp_name, stage) DO NOTHING;
```

(b) FDP publishes a source row + readiness — run by/for each FDP:

```
INSERT INTO control.pipeline_run_source_fdp
  (effective_month, cdp_name, fdp_name, source_table_name, source_partition,
   source_row_count, fdp_ready, fdp_ready_timestamp, created_timestamp, updated_timestamp)
VALUES (@effective_month, @cdp_name, @fdp_name, @table, @partition,
        @row_count, TRUE, now(), now(), now())
ON CONFLICT (effective_month, cdp_name, fdp_name, source_table_name, source_partition)
DO UPDATE SET source_row_count = EXCLUDED.source_row_count,
              fdp_ready = EXCLUDED.fdp_ready,
              fdp_ready_timestamp = EXCLUDED.fdp_ready_timestamp,
              updated_timestamp = now();
```

(c) Readiness gate — `CDP_BUILD_READINESS_CHECK` (RC 0 -> proceed):

```
-- Completeness + readiness vs the reference table. Empty result = all required sources exist & ready
 ↪  -> RC 0.
SELECT r.fdp_name, r.source_object,
       CASE WHEN a.cdp_name IS NULL THEN 'MISSING' ELSE 'NOT_READY' END AS issue
FROM   control.cdp_source_reference r
LEFT JOIN control.pipeline_run_source_fdp a
       ON a.cdp_name = r.cdp_name AND a.fdp_name = r.fdp_name
      AND a.source_table_name = r.source_object AND a.effective_month = @effective_month
WHERE  r.cdp_name = @cdp_name AND r.active AND r.required
  AND  (a.cdp_name IS NULL OR a.fdp_ready = FALSE);
-- job exits 0 iff this returns no rows (nothing missing, nothing not-ready)
```

(d) Atomic stage claim — `STARTED` (see Section 5.7; one statement, race-safe):

```
UPDATE control.pipeline_run
   SET status='STARTED', start_timestamp=now(), updated_timestamp=now()
 WHERE effective_month=@effective_month AND cdp_name=@cdp_name AND stage=@stage AND status='NOT_
  ↪  STARTED'
RETURNING stage;     -- 1 row = claimed, 0 rows = already running/done → back off
```

(e) CDP_BUILD complete — roll up child counts + record mart outputs:

```
UPDATE control.pipeline_run p
   SET status='COMPLETED', end_timestamp=now(), updated_timestamp=now(),
       cdp_table_name=@cdp_table_name, cdp_row_count=@cdp_row_count,
       source_total_row_count=(SELECT COALESCE(sum(source_row_count),0)
                                 FROM control.pipeline_run_source_fdp c
                                WHERE c.effective_month=p.effective_month AND c.cdp_name=p.cdp_name)
 WHERE p.effective_month=@effective_month AND p.cdp_name=@cdp_name AND p.stage='CDP_BUILD' AND
  ↪  p.status='STARTED';
```

(f) TRANSFORMATION complete:

```
UPDATE control.pipeline_run
   SET status='COMPLETED', end_timestamp=now(), updated_timestamp=now(),
       transformation_job_name=@job_name, output_storage_path=@gcs_path
 WHERE effective_month=@effective_month AND cdp_name=@cdp_name AND stage='TRANSFORMATION' AND
  ↪  status='STARTED';
```

(g) Mark ERROR (any stage):

```
UPDATE control.pipeline_run
   SET status='ERROR', end_timestamp=now(), updated_timestamp=now(), error_message=@message
 WHERE effective_month=@effective_month AND cdp_name=@cdp_name AND stage=@stage AND status='STARTED';
```

(h) Reconciliation check (post-build / post-transform):

```
-- alert if any boundary differs beyond tolerance
SELECT source_total_row_count, cdp_row_count,
       abs(source_total_row_count - cdp_row_count) AS delta
FROM control.pipeline_run
WHERE effective_month=@effective_month AND cdp_name=@cdp_name AND stage='CDP_BUILD';
-- CSV row count is compared by TRANSFORMATION_OUTPUT_VERIFY against cdp_row_count (post-transform
 ↪  expectations)
```

13. CDP_BUILD — DBT in Docker (implementation)

13.1 The mapping sheet & incremental field population

The CDP is **contract-first**: every column it will ever expose is fixed up front from the **mapping sheet** — the source-to-target specification mapping each CDP field to its FDP origin. The CDP is built against that complete schema from the outset. Columns whose FDP source is not yet available are emitted as typed `NULL` placeholders, so the CDP's shape — and the downstream CSV contract — is stable immediately; only the population grows over time. As FDPs expose new fields, the mapping sheet status is updated and the dbt model amended — never the CDP schema.

> **Two mappings, don't conflate them** (both pivot around the *fixed* CDP schema): **Mapping sheet** = FDP field → CDP field, governs the **dbt build** (this section). **`mapping_template`** (Section 14.1) = CDP field → CSV column, governs the **Dataflow output**. Because the CDP schema is fixed, this template stays stable too.

Mapping sheet — one row per CDP field. Governed artifact (versioned; ideally a dbt seed so the model can reference it):

Column	Meaning
`cdp_field` / `cdp_type`	Target CDP column and type (**fixed**)
`source_fdp` / `source_table` / `source_field`	FDP origin of the value
`transform_rule`	direct, cast, derivation, seed-lookup, default
`status`	`available` (mapped now) or `pending` (FDP not yet exposing it)
`populated_from_version`	build version the field went live

Example (`cdp_customer_360`):

cdp_field	cdp_type	source_fdp	source_field	transform_rule	status
customer_id	STRING	fdp_party	party_id	direct	available
segment	STRING	fdp_party	segment_code	seed-lookup	available
region	STRING	fdp_account	region	direct	available
lifetime_value	NUMERIC	fdp_txn	ltv_12m	sum	**pending**
churn_score	FLOAT	fdp_ml	score	direct	**pending**

dbt pattern. The mart model's SELECT lists **all** CDP columns: `available` rows emit their source expression; `pending` rows emit `CAST(NULL AS <cdp_type>) AS <cdp_field>`. `schema.yml` declares the full column set, with `not_null` only on the key/available columns. When an FDP field lands: flip `status` to `available`, replace the `NULL` with the mapping, bump `populated_from_version`, ship — no schema change, no downstream break.

```
SELECT
  p.party_id                          AS customer_id,      -- available
  seg.segment_name                    AS segment,          -- available (seed lookup)
  a.region                            AS region,           -- available
  CAST(NULL AS NUMERIC)               AS lifetime_value,   -- pending: fdp_txn.ltv_12m
  CAST(NULL AS FLOAT64)               AS churn_score        -- pending: fdp_ml.score
FROM {{ ref('stg_party') }} p
LEFT JOIN {{ ref('stg_account') }} a USING (customer_id)
LEFT JOIN {{ ref('seg_map') }} seg ON seg.code = p.segment_code
```

Project layout (one dbt project, marts selected per `cdp_name`):

```
dbt_cdp/
  dbt_project.yml
  profiles.yml                # BigQuery, creds via env / Workload Identity
  models/
    staging/                  # 1:1 with FDP sources, light cleaning
      _fdp_sources.yml        # `sources:` = the FDP tables (contract)
      stg_party.sql ...
    marts/
      cdp_customer_360/       # one folder per mart; tag = cdp_name
        cdp_customer_360.sql  # the mart model (incremental, partitioned)
        _cdp_customer_360.yml # schema tests + contract
  macros/ tests/ seeds/
```

Partitioned, incremental mart model (`effective_month` is a run var):

```
{{ config(materialized='incremental', incremental_strategy='insert_overwrite',
          partition_by={'field':'effective_month','data_type':'date','granularity':'month'},
          cluster_by=['customer_id','segment','region'],
          require_partition_filter=true, tags=['cdp_customer_360']) }}

SELECT DATE_TRUNC(DATE('{{ var("effective_month") }}'), MONTH) AS effective_month,
       p.customer_id, p.segment, a.region /* ... */
FROM   {{ ref('stg_party') }}  p
JOIN   {{ ref('stg_account') }} a USING (customer_id)
{% if is_incremental() %}
  WHERE DATE_TRUNC(DATE('{{ var("effective_month") }}'), MONTH)
        = DATE_TRUNC(DATE('{{ var("effective_month") }}'), MONTH)   -- overwrite just this month's
        ↪ partition
{% endif %}
```

Docker invocation — the scheduler runs the pinned image with the two params; `--select` picks the mart, `--vars` passes the month:

```
docker run --rm \
  -e DBT_PROJECT=cdp -e GOOGLE_CLOUD_PROJECT=$PROJECT \
  -v /secrets:/secrets:ro \
  $REGION-docker.pkg.dev/$PROJECT/cdp/dbt-cdp@sha256:<digest> \
  bash -lc "dbt deps && \
            dbt run  --select tag:${CDP_NAME} --vars '{cdp_name: ${CDP_NAME}, effective_month:
↪  ${EFFECTIVE_MONTH}}' && \
```

```
            dbt test --select tag:${CDP_NAME} --vars '{cdp_name: ${CDP_NAME}, effective_month:
↪  ${EFFECTIVE_MONTH}}'"
# exit 0 = run+tests passed → caller writes COMPLETED; non-zero → caller writes ERROR + captured log
↪  tail
```

Tests that gate the build (in `_*.yml`): `not_null` + `unique` on the mart grain key, `relationships` back to staging, `accepted_values` on enums, plus a **custom row-count reconciliation test** comparing the mart count to `SUM(source_row_count)` within tolerance. A failing `dbt test` must fail the job (non-zero exit) so the stage goes ERROR and transformation never runs.

Capturing outputs: after a green run, `CDP_BUILD_ROWCOUNT` reads `SELECT COUNT(*) FROM mart.cdp_<name> WHERE effective_month=@m` and the resolved table name, then statement (e) writes them to metadata.

14. TRANSFORMATION — Apache Beam (Java) on Dataflow

Pipeline options (the three params + derived paths) — a `PipelineOptions` interface:

```
public interface CdpTransformOptions extends DataflowPipelineOptions {
  @Description("Target mart") @Validation.Required String getCdpName();      void setCdpName(String
  ↪  v);
  @Description("YYYY-MM-01")  @Validation.Required String getEffectiveMonth(); void
  ↪  setEffectiveMonth(String v);
  @Description("gs:// path to mapping template JSON") @Validation.Required
                              String getMappingTemplate();                   void
                               ↪  setMappingTemplate(String v);
  @Description("Processing dataset") @Default.String("proc") String getProcessingDataset(); void
  ↪  setProcessingDataset(String v);
  @Description("gs:// output prefix") @Validation.Required String getOutputPath(); void
  ↪  setOutputPath(String v);
}
```

Pipeline shape (read one partition → map via template → stage → read-back/aggregate → CSV + markers):

```
Pipeline p = Pipeline.create(opts);
MappingTemplate tpl = MappingTemplate.loadAndValidate(opts.getMappingTemplate(), martSchema); // fail
↪  fast

PCollection<TableRow> rows = p.apply("ReadMart", BigQueryIO.readTableRows()
    .fromQuery(String.format(
       "SELECT * FROM `%s.mart.cdp_%s` WHERE effective_month = DATE('%s')",
       project, opts.getCdpName(), opts.getEffectiveMonth())).usingStandardSql());

PCollection<TableRow> mapped = rows.apply("ApplyTemplate", ParDo.of(new ApplyMappingFn(tpl)));

mapped.apply("StageIntermediate", BigQueryIO.writeTableRows()
    .to(String.format("%s.%s.cdp_%s_%s_step1", project, opts.getProcessingDataset(),
        opts.getCdpName(), yyyymm(opts)))
    .withCreateDisposition(CREATE_IF_NEEDED).withWriteDisposition(WRITE_TRUNCATE)); // idempotent

PCollection<String> csv = readBackAndAggregate(p, tpl, opts)   // join/aggregate from proc
    .apply("ToCsv", MapElements.via(new RowToCsvFn(tpl)));

csv.apply("WriteCsv", TextIO.write().to(opts.getOutputPath() + "part")
    .withSuffix(".csv").withHeader(tpl.csvHeader()).withNumShards(0)); // runner-chosen sharding
// On success: write _SUCCESS + manifest.json (row count, shard list, template version) to outputPath
p.run().waitUntilFinish();
```

Build & launch (Flex Template — default):

```
# build once per release (CI): shaded jar → Flex Template spec in GCS
mvn -q -Pdataflow-runner clean package
gcloud dataflow flex-template build gs://$ENV-cdp-templates/cdp-transform.json \
  --image-gcr-path $REGION-docker.pkg.dev/$PROJECT/cdp/cdp-transform:$TAG \
  --sdk-language JAVA --jar target/cdp-transform-bundled.jar \
  --env FLEX_TEMPLATE_JAVA_MAIN_CLASS=com.acme.cdp.TransformMain
```

```
# launch per run (scheduler TRANSFORMATION_DATAFLOW_LAUNCH):
gcloud dataflow flex-template run "df-${CDP_NAME}-${YYYYMM}" \
  --template-file-gcs-location gs://$ENV-cdp-templates/cdp-transform.json \
  --region $REGION --service-account-email sa-cds-pipeline@$PROJECT.iam.gserviceaccount.com \
  --parameters cdpName=${CDP_NAME},effectiveMonth=${EFFECTIVE_MONTH},\
mappingTemplate=gs://$ENV-cdp-config/${MAPPING_TEMPLATE},\
outputPath=gs://$ENV-cdp-output/${CDP_NAME}/${YYYY_MM}/
```

`TRANSFORMATION_DATAFLOW_WAIT` then polls `gcloud dataflow jobs describe <id> --format='value(currentState)'` until `JOB_STATE_DONE` (success) / `FAILED` / `CANCELLED`.

14.1 `mapping_template` — CDP-mart → Mainframe-file mapping

This is the spec that maps the **CDP mart columns (the effective SELECT)** to the **file we write to GCS for the Mainframe**. It is a **versioned YAML** file in `gs://<env>-cdp-config/` (JSON is also accepted; **YAML is preferred** because it is human-maintained alongside the mapping sheet). It is loaded and validated against the mart schema at job start, so a bad template fails the job before any write.

The job derives its query from the template — conceptually `SELECT <source/expression cols> FROM mart.cdp_<name> WHERE effective_month = @m AND <filters>` — then formats each row per the `output` block:

```yaml
template_version: v3
source:
  partition_column: effective_month          # the pruned read (Section 6.8)
output:
  format: delimited                          # delimited | fixed_width
  delimiter: ","
  quote: "\""
  header: true                               # column-name header record
  trailer: true                              # record-count trailer record
  null_value: ""
  encoding: UTF-8                            # or a Mainframe codepage, e.g. IBM-1047 (EBCDIC)
  line_terminator: "LF"                      # LF | CRLF
columns:                                      # order = output column order
  - target: CUSTOMER_ID                       # CSV / record field name
    source: customer_id                       # mart column  (the SELECT)
    type: STRING
    required: true
    width: 18                                 # fixed_width only
    align: left                               # alphanumeric -> left-justified
    pad_char: " "                             # ... space-filled on the right
  - target: SEGMENT
    source: segment
    type: STRING
    width: 4
    align: left
    pad_char: " "
  - target: MONTH_END
    expression: "LAST_DAY(effective_month)"   # SQL evaluated on the mart row
    type: DATE
    format: "yyyyMMdd"
    width: 8
    align: left
  - target: BALANCE
    source: balance
    type: DECIMAL
    format: "0.00"
    width: 13
    align: right                              # numeric -> right-justified
    pad_char: "0"                             # ... zero-filled on the left
    sign: leading                             # sign convention per copybook
filters:
  - "status = 'ACTIVE'"
```

Each entry maps a **mart column (`source`) or SQL `expression`** to an **output field (`target`)**, in declared order, with type/format and — for `fixed_width` — `width`/`align`/`pad_char`/`sign`. `align` and `pad_char` default by type (see Section 14.2) but are set explicitly per column so the layout matches the target copybook exactly. `ApplyMappingFn` is generic and entirely template-driven, so a new output layout is a config change (new `template_version`), never a code change. The `template_version` is written into `manifest.json` for lineage. Because the CDP schema is fixed (Section 13.1), this template is stable too.

14.2 Mainframe output contract (what lands in GCS)

The transform writes the file(s) to `gs://<env>-cdp-output/<cdp>/<YYYY-MM>/`; the Mainframe ingests from there (pulled, or moved by a managed-file-transfer agent).

Agreed transfer format — human-readable CSV. We hand off a **human-readable, comma-`delimited` CSV in UTF-8/ASCII**. We deliberately **do not** produce EBCDIC or fixed-width on our side; the **Mainframe converts to its internal (non-readable) format** — EBCDIC codepage, fixed-width per copybook, zero-padding/justification, packed/zoned decimals and sign handling — as its own step **before** it ingests. This keeps the GCS hand-off simple, inspectable, and easy to reconcile, and keeps copybook/codepage concerns where the copybook lives (the Mainframe). Fixed-width/EBCDIC remain *supported* options in the template if ever needed, but are **not used** for this interface.

What therefore remains **our** responsibility (the readable-CSV contract): column order, `delimiter`, `quote`, optional header row, a record-count **trailer**, `line_terminator`, and the null representation. Alignment, padding (spaces vs zeroes), and codepage are **the Mainframe's** responsibility at conversion — they do not apply to the readable CSV we deliver.

- **Record format** — `delimited` (CSV, comma-separated, with `delimiter`/`quote`). Because it is delimited and readable, fields are **not** padded or aligned — there is no left/right justification or spaces-vs-zeroes question on our side.
- **Encoding** — `UTF-8`/ASCII (readable). EBCDIC conversion happens on the Mainframe, not here.
- **Header / trailer** — optional column-name header and a record-count **trailer** (control total) for the Mainframe to reconcile on load — this trailer count is also the CSV count used in the reconciliation chain (R9).
- **Line terminator** — LF or CRLF per the consumer.
- **Nulls, decimals, dates** — readable values per `null_value` (empty by default), and `format` for numbers/dates (e.g. `0.00`, `yyyyMMdd`, sign shown as a readable `-`). The Mainframe applies any packing/zoned-decimal/sign encoding during its conversion.
- **Sharding vs single file** — by default the output is sharded; see Section 14.3 for the knobs and the single-file option.

14.3 Output sharding

Beam writes the output as **multiple part files in parallel** — one writer per bundle/worker — rather than streaming everything through a single writer. This is what gives the write its throughput and keeps it memory-safe; the shards together are the deliverable, gated by a _SUCCESS marker.

Naming. `part-00000-of-000NN.csv`, `part-00001-of-000NN.csv`, … plus `_SUCCESS` and `manifest.json` (the shard list and per-shard + total row counts) in the same `output_storage_path`.

The knobs:

Knob	Effect	Guidance
`numShards = 0`	Runner chooses shard count (scales with workers)	**Default** — best throughput for large CDPs
`numShards = N`	Fixed number of output files	Use when a consumer expects a known file count
`numShards = 1`	One file (a single final writer)	Small CDPs only — it serialises the write, so avoid for large data
Target shard size	Drives N when fixing it	Aim ~128–512 MB/shard: `N ~ ceil(total_output_bytes / target_shard_bytes)`
`withHeader`	Header record per file	With sharding, the header is written **per shard** — see single-file note below

When the Mainframe needs one dataset. Two options: (a) set `numShards = 1` for small CDPs; or (b) keep parallel sharding and **concatenate** the shards into one file at hand-off — `gsutil compose` (GCS-side) or a short final merge

step — writing a **single header and a single record-count trailer** for the combined file (don't carry per-shard headers into the merged dataset). The reconciliation trailer count is the sum across shards.

Ordering. Order is not guaranteed *across* shards. If the consumer needs a global sort, either sort then write a single shard, or sort-then-merge at hand-off (extra cost); Mainframe load utilities usually don't require cross-shard order.

Safety. Consumers gate on `_SUCCESS` + metadata `COMPLETED`, never on the presence of individual `part-*` files (R7); on re-run the output path is overwritten, so the same shard set is reproduced (idempotent).

14.4 Processing dataset & intermediaries

> **Not to be confused with dbt staging.** These are the **Dataflow transform's** working tables in the `proc` dataset (TRANSFORMATION stage, Data Mart -> CSV). They are **distinct from dbt staging models** (`stg_*`, Section 13), which belong to the CDP_BUILD stage (FDP -> Data Mart). To keep the terms apart, the transform's tables are named `_step1`/`_step2` and the word "staging" is reserved for dbt.

The transform stages its work in the **processing dataset** as a short chain of intermediate tables, named `proc.<cdp>_<YYYYMM>_<step>`:

Step	Table	Purpose
Step 1	`proc.<cdp>_<YYYYMM>_step1`	Mapped/normalised rows from the mart read (template applied, filters pushed down)
Step 2	`proc.<cdp>_<YYYYMM>_step2`	Joined / aggregated intermediate (keyed work; heavy joins done in BigQuery, read back in splits)
Final	*(no table)*	Formatted records written straight to GCS per the `output` contract

Conventions: every intermediate is written `WRITE_TRUNCATE` (so a retry starts clean), carries a **partition/table expiration (default 14 days)**, and is namespaced by `cdp_name+effective_month` so parallel CDPs never collide (Section 4.4). Keeping intermediates as real tables — rather than only in-stream — isolates transform scratch from the curated mart, lets heavy joins run as set-based BigQuery SQL, and makes each run's steps inspectable for debugging.

Dataset vs tables — who creates what. The **`proc` dataset** (the container) is **not** created by the Dataflow job; it is provisioned once as **infrastructure** (Terraform, Phase 0) with its location, default table expiration (TTL), labels, and a grant of `bigquery.dataEditor` to `sa-cds-pipeline`. The Dataflow job only creates and replaces the **tables inside it at runtime** via `BigQueryIO.write` with `CREATE_IF_NEEDED` + `WRITE_TRUNCATE`. This split is deliberate: dataset creation is a higher-privilege, location-/policy-sensitive action that belongs in IaC (and within the VPC-SC perimeter), while the job runs with least privilege — it can write tables in `proc` but cannot create datasets. The same applies to the `mart` dataset (created by IaC; tables written by dbt).

15. Scheduler (TWS) Job Definitions

A monthly **driver** (`CDP_FANOUT`) enumerates the active CDPs from the registry and submits one run-cycle **per `cdp_name` in parallel**, bounded by `max_concurrency` (Section 4.4). The per-CDP run-cycle below is therefore instantiated N times concurrently — the jobs are identical, parameterised by `cdp_name/effective_month`. Each job runs a small wrapper script (`psql`, `docker/gcloud run`, or `gcloud dataflow`); **the metadata predicate, not the TWS link, is authoritative** — every job re-checks state on entry. Standard return codes: `0` = success/proceed, `1` = business stop (e.g. not ready — wait/retry next tick), `2` = error (write `ERROR`, alert).

Job	Command (wrapper)	Key params	RC 0 means	Predecessor	On failure
`CDP_FANOUT`	driver: list CDPs, submit per-CDP cycles	month, max_concurrency	cycles launched	monthly trigger	partial launch → alert, others proceed
`CDP_BUILD_READINESS_CHECK`	`psql` Section 12(c)	cdp, month	all FDPs ready	fan-out (per CDP)	RC1 → re-arm next tick; alert at cutoff
`CDP_BUILD_META_STARTED`	`psql` Section 12(d) stage=CDP_BUILD	cdp, month	stage claimed	readiness	RC0 rows=0 → already running, skip
`CDP_BUILD_DBT_RUN`	`docker run … dbt run` Section 13	cdp, month	models built	build started	RC2 → `CDP_BUILD_META_ERROR`
`CDP_BUILD_DBT_TEST`	`docker run … dbt test`	cdp, month	all tests pass	dbt run	RC2 → `CDP_BUILD_META_ERROR`

Job	Command (wrapper)	Key params	RC 0 means	Predecessor	On failure
CDP_BUILD_ROWCOUNT	bq query count	cdp, month	count captured	dbt test	RC2 → error
CDP_BUILD_META_COMPLETED	psql Section 12(e)	cdp, month, table, count	row updated	rowcount	RC2 → error
TRANSFORMATION_GATE	psql Section 5.7 gate	cdp, month	gate TRUE	build completed	RC1 → wait
TRANSFORMATION_META_STARTED	psql Section 12(d) stage=TRANSFORMATION	cdp, month	stage claimed	gate	rows=0 → skip
TRANSFORMATION_DATAFLOW_LAUNCH	gcloud … flex-template run Section 14	cdp, month, template	job submitted	TRANSFORMATION started	RC2 → TRANSFORMATION_META_ERROR
TRANSFORMATION_DATAFLOW_WAIT	poll jobs describe	job id	state=DONE	launch	FAILED/CANCELLED → error
TRANSFORMATION_OUTPUT_VERIFY	check _SUCCESS + count vs cdp_row_count	gcs path	output valid	wait	mismatch → error
TRANSFORMATION_META_COMPLETED	psql Section 12(f)	cdp, month, job, path	row updated	verify	RC2 → error
<STAGE>_ALERT_*	notify (ServiceNow incident / Ops)	context	alert sent	any error edge	—

Calendar & fan-out: CDP_FANOUT fires on a monthly schedule after the FDP cutoff and launches the per-CDP cycles in parallel (capped by max_concurrency). Within each cycle, CDP_BUILD_READINESS_CHECK re-arms on a short interval until ready or the SLA deadline triggers CDP_BUILD_ALERT_NOT_READY. CDP cycles are independent — one stalling never blocks another.

16. Parallel Execution & Resource Management

Section 4.4 defines the concurrency *model*. This section states what must be built to run CDPs in parallel and how the shared Google Cloud resources are sized and managed.

16.1 What to build for parallel runs

- **CDP registry (`control.cdp_registry`).** The CDP-level config table listing the CDPs active each month: cdp_name, enabled, priority, resource_class (S/M/L, sets per-CDP worker/slot ceilings), mapping_sheet_version, and **has_transformation** (whether the CDP runs the TRANSFORMATION stage). The driver reads it to know what to launch — and, via has_transformation, **whether to seed a TRANSFORMATION stage row at all**:

```
CREATE TABLE control.cdp_registry (
  cdp_name           TEXT PRIMARY KEY,
  enabled            BOOLEAN NOT NULL DEFAULT TRUE,
  priority           INT     NOT NULL DEFAULT 100,
  resource_class     TEXT    NOT NULL DEFAULT 'M',   -- S | M | L
  has_transformation BOOLEAN NOT NULL DEFAULT TRUE,  -- FALSE = build-only CDP
  mapping_template   TEXT,                            -- required when has_transformation
  mapping_sheet_version TEXT,
  updated_timestamp  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

  A **build-only CDP** (has_transformation = FALSE) has no mapping_template and no TRANSFORMATION stage row; it completes after CDP_BUILD and is consumed directly from the Data Mart.

- **Driver (`CDP_FANOUT`).** Reads the registry, seeds the metadata rows (Section 12a) for each CDP, and submits the per-CDP cycle for each — subject to a concurrency limit. It records launch status and continues past any single CDP that fails to launch.
- **Concurrency control.** A max_concurrency bound caps how many CDP cycles are active at once; a work queue releases the next CDP as one finishes. max_concurrency = 1 runs the CDPs **strictly sequentially** (one after another) — a valid, low-resource operating mode — while higher values run them in parallel. Optionally apply per-resource_class sub-limits (e.g. at most two L CDPs at a time). TWS's own job-stream limit is an equivalent lever if concurrency is enforced in the scheduler rather than the driver.

- **Isolation prerequisites (already in the design — verify as a checklist).** Per-CDP metadata rows, mart table (`mart.cdp_<name>`), processing tables (`proc.<cdp>_<YYYYMM>_<step>`, `WRITE_TRUNCATE`), Dataflow job (`df-<cdp>-<YYYYMM>`), and GCS output prefix; per-stage service accounts. No globally-named temp objects.
- **Idempotency & failure isolation under load.** The atomic per-`(cdp,month,stage)` claim prevents double-starts; re-seeding is idempotent (`ON CONFLICT DO NOTHING`), so the driver is safe to restart. One CDP reaching `ERROR` never blocks the others; alerts are raised per CDP.

16.2 Resource management & sizing

Parallelism does not change the *total* work; it concentrates it in time. Resourcing must therefore cover the **peak** (concurrent) demand, not the monthly average. Each shared resource and how to manage it:

Resource	Limit / unit	Scales with	Management approach
BigQuery compute	reservation **slots**	concurrent builds + queries	Dedicated reservation + assignment for the build window; autoscaling max >= peak; isolate from interactive workloads
BigQuery storage	bytes	total mart history	Monthly partitioning + TTL; partition pruning controls scan cost
Dataflow workers	regional **vCPU** quota, **in-use IP** quota	concurrent transform jobs x workers/job	Raise vCPU quota; cap `maxNumWorkers` per job; use no-public-IP (Private Google Access) to avoid the IP-quota ceiling
Cloud SQL (control)	`max_connections`, CPU/mem	concurrent jobs x connections/job	Connection pooling (PgBouncer / Cloud SQL Auth Proxy) with a fixed ceiling; right-size tier
Concurrency	`max_concurrency`	—	The master lever; tune to stay within the binding quota above
GCS output	request rate / egress	output volume	Effectively unbounded at this scale; standard

Sizing approach. Let **C** = `max_concurrency` (peak concurrent CDP cycles). Size each resource to C, with headroom:

- **BigQuery:** `peak_slots ~ C x slots_per_build`. Provision the reservation (or autoscale max) at or above this.
- **Dataflow:** `peak_vCPU ~ C x maxNumWorkers x vCPU_per_worker`; ensure the regional vCPU quota covers it. Use no-public-IP workers to remove the in-use-IP ceiling.
- **Cloud SQL:** `peak_connections ~ C x jobs_active_per_cycle x conns_per_job`; pool to a fixed ceiling below `max_connections`.
- Set **C = min over resources of (available_quota / per-CDP demand)**, then back off for headroom.

Worked example — 60 CDPs, monthly, target < 6 h. With `C = 12`: BigQuery ~ 12 x 300 = 3,600 slots → provision a 4,000-slot reservation; Dataflow ~ 12 x 20 workers x 4 vCPU = 960 vCPU → raise the regional quota to ~1,200 and run no-public-IP; Cloud SQL ~ 12 x ~3 connections ~ 36 → pool ceiling 100 on a mid-tier instance. Throughput: 60 / 12 ~ 5 waves; at ~50 min/cycle ~ 4.2 h, inside the SLA.

Operational levers. Raise C for speed (more peak resource, higher cost) or lower it for safety/cost; `C = 1` runs CDPs strictly sequentially (lowest peak resource) and is a sensible starting mode before scaling up. Order by `priority` so critical CDPs run first; throttle large CDPs via `resource_class`; widen or shift the build window; keep a separate reservation for build vs. interactive analytics. Monitor concurrent jobs, slot utilisation, worker count, and connection count, and alert as any approaches its quota (risk R4).

17. IAM & Service Account

A **single pipeline service account** (`sa-cds-pipeline`) runs the whole pipeline. This is a deliberate decision: all marts (CDPs) are inputs to one **CDS pre-processing** flow, so the pipeline operates as a single trust domain, and one account keeps access management and operations simple. It is used by the TWS wrappers, the dbt build container, and the Dataflow workers.

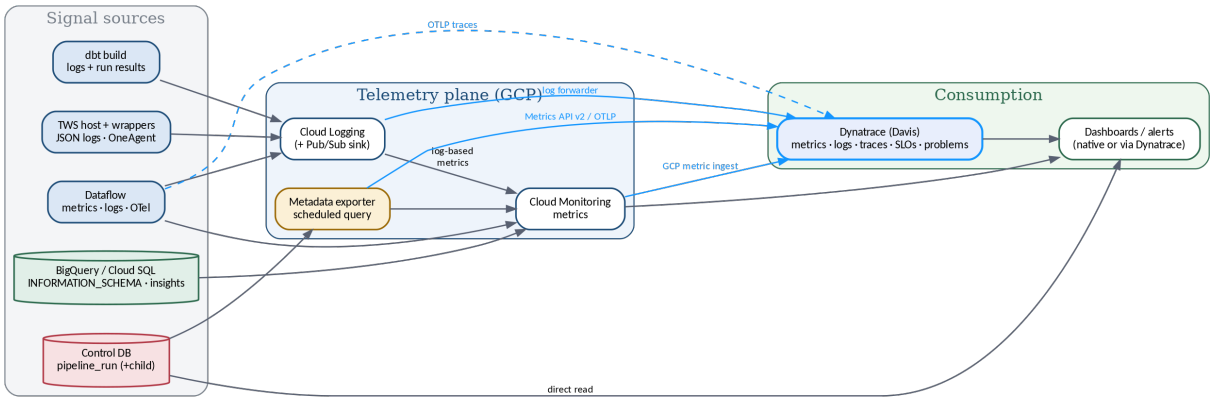
Service account	Used by	Roles (scoped — never project-wide)
`sa-cds-pipeline`	TWS wrappers, dbt container, Dataflow workers	`roles/bigquery.dataViewer` on `fdp.*`; `roles/bigquery.dataEditor` on `mart.*` and `proc.*`; `roles/bigquery.jobUser`; `roles/storage.objectAdmin` on the output / config / template buckets; `roles/cloudsql.client` (control DB); `roles/dataflow.developer` + `roles/iam.serviceAccountUser` on itself (to launch and run Dataflow)

The single account is kept safe by compensating controls: roles are scoped to the specific FDP/mart/proc datasets and buckets (not project-wide); **Workload Identity** is used (no long-lived keys); a **VPC-SC** perimeter encloses BigQuery, GCS, and Cloud SQL; and the account is rotated with full audit logging. The concentration trade-off is tracked as R3.

Human access is separate and governed. `sa-cds-pipeline` is a *machine* identity — people never use it. Developers/testers/analysts who need to query the BigQuery data directly request **least-privilege, time-bound** access through the standard access-request workflow, and **PII columns are a separate, policy-tag-gated grant**. Full detail in Section 24 (Data Classification, PII & Access Governance).

18. Observability

Observability is **designed in, not bolted on** — the control plane already records the state of every run, so the strategy is to treat that as the backbone and layer GCP-native logs, metrics, dashboards, and alerts on top.



18.1 The control plane is the observability backbone

Because every stage writes its status, timestamps, counts, lineage, and errors to `control.pipeline_run` (+ child), the system's *business* state is already fully observable from one queryable place — no log scraping required to answer "what is the state of this month's run?". The three pillars are: (1) the **control DB** for run/business state, (2) **Cloud Logging** for detailed execution traces, and (3) **Cloud Monitoring** for time-series metrics and alerting. A small **metadata exporter** (a scheduled query) publishes control-DB-derived values as Cloud Monitoring custom metrics so business signals and platform signals sit side by side.

18.2 Metrics catalogue

Metric	Source	Why it matters
Stage status counts (by `status`)	Control DB → custom metric	Live run progress; count in ERROR
Stage duration (`end-start`)	Control DB	SLA tracking; anomaly detection
Stuck-stage age (`now-start` while STARTED)	Control DB	Detects orphaned runs (R1)
Reconciliation delta (source→mart→CSV)	Control DB	Primary correctness signal (R9)
FDP readiness lead-time vs cutoff	Child table	Upstream-SLA / late-FDP risk (R5)
Concurrent CDP cycles / Dataflow jobs	Monitoring	Parallelism vs quota (R4)
BigQuery slots & bytes scanned	BigQuery `INFORMATION_SCHEMA.JOBS`	Cost & saturation (R4, R10)
Dataflow vCPU-hours / throughput / lag	Dataflow metrics	Transform performance & cost
Cloud SQL connections / CPU	Cloud SQL Insights	Control-store headroom

18.3 Logs & correlation

Every wrapper, dbt run, and Dataflow job emits **structured JSON** logs carrying a correlation key — `run_id = cdp_name + effective_month + stage` — plus `status` and `job_id`. This makes a single run traceable end-to-end across components in Cloud Logging, and **log-based metrics** (e.g. error counts, specific failure signatures) feed Monitoring. dbt `run_results.json` and the Dataflow job id are linked from the metadata row for one-click drill-down.

18.4 Dashboards

- **Run board** — a per-month grid of CDP x stage with colour-coded status, durations, and row counts (reads the control DB directly via Looker Studio / Looker).
- **SLA & throughput** — completion progress vs deadline, stage-duration distributions, late-FDP list.
- **Resources & cost** — concurrent jobs, BigQuery slot utilisation and bytes scanned, Dataflow workers, Cloud SQL connections, all against quota.

18.5 Alerting

Alert	Condition	Routes to
Stage error	any row `status='ERROR'`	ServiceNow incident / Ops
Stuck stage	`STARTED` age > expected duration	Ops (run book 3.5)
Reconciliation breach	delta beyond tolerance	Data platform + hold output
FDP late	not all `fdp_ready` by cutoff	FDP owner + platform
Quota pressure	slots/workers/connections near limit	Platform (lower `max_concurrency` or raise quota)
Dataflow failed	job state `FAILED/CANCELLED`	Ops

18.6 SLOs

Define and track, per month: **completion** (all CDPs `COMPLETED` by the deadline), **per-stage duration** (P95 within budget), **reconciliation pass rate** (100% before any output is released), and **freshness** (output available within N hours of FDP readiness). SLO burn drives the alerting thresholds above. The **full SLI/SLO/SLA definitions and targets are in Section 23**; this section is the measurement hook for them.

18.7 Audit & lineage

`created_timestamp/updated_timestamp` give a basic audit trail; for full history, append each transition to a `pipeline_run_history` table (or rely on Cloud SQL PITR + the update audit log). Lineage — which FDP tables/partitions and which `mapping_template` / `mapping_sheet` version produced an output — is already captured in the child table and `manifest.json`, satisfying "how was this CSV produced?" for governance.

18.8 Integrating Dynatrace

Dynatrace plugs in **at the telemetry plane** — it becomes an additional consumer of the same signals, so nothing in the pipeline changes. There are five ingestion paths, each mapping to a signal we already produce:

Signal	Dynatrace ingestion path	Notes
GCP platform metrics (BigQuery slots/bytes, Dataflow, Cloud SQL, GCS)	**Native Google Cloud integration** (Dynatrace pulls Cloud Monitoring metric ingest)	Zero code; covers the managed services
Business / run metrics (stage status, durations, stuck-age, reconciliation delta, FDP lead-time)	**Metrics API v2** or **OTLP** from the metadata exporter	The exporter posts control-DB-derived metrics straight to Dynatrace as well as Cloud Monitoring
Logs (dbt, Dataflow, wrappers)	**Dynatrace GCP log forwarder** (Cloud Function/Dataflow reading a Pub/Sub log sink → Log Ingest)	Preserve the `run_id` key so logs correlate to metrics/traces
Traces (end-to-end run)	**OpenTelemetry (OTLP)** from the Java Beam job and wrapper scripts	A run becomes a distributed trace: build span → transform span
Host / infra (TWS scheduler, GKE nodes if used)	**OneAgent**	Managed services (Dataflow, Cloud Run, Cloud SQL) are covered by the GCP integration, not OneAgent

Once the signals are in, Dynatrace provides the **consumption layer**: dashboards, **Davis** automatic problem detection/anomaly alerting (routed to ServiceNow / Ops), and **SLOs** (the Section 18.6 targets defined as Dynatrace SLOs).

It can either augment or replace the native Cloud dashboards/alerts. Recommended approach: start with the **native GCP integration + Metrics API** for platform and business signals (fastest value), then add **log forwarding** and **OTel tracing** for full correlation, and **OneAgent** on the TWS host for infra. Because Dynatrace consumes the existing `run_id`-keyed signals, metrics, logs, and traces line up on a single run with no bespoke glue.

19. Idempotency, Retry & Restart Runbook

- **Re-running a `COMPLETED` stage is a no-op** — the gates (`status='NOT_STARTED'` claim; `BUILD=COMPLETED AND TRANSFORMATION=NOT_STARTED`) make sure of it. Safe to replay any job.
- **Recover a failed build/transform:** fix the cause, then reset and let the scheduler re-pick:

```
UPDATE control.pipeline_run SET status='NOT_STARTED', error_message=NULL, updated_
↪  timestamp=now()
WHERE effective_month=@m AND cdp_name=@c AND stage=@stage AND status='ERROR';
```

- **Dataflow restart is safe:** intermediate tables are `WRITE_TRUNCATE` and the output path is overwritten, so a re-launch reproduces the same result. Consumers gate on `_SUCCESS` + metadata `COMPLETED`, never on partial files.
- **Stuck `STARTED`:** the stuck-job detector flags rows in `STARTED` past the SLA; operator confirms the underlying job is dead, then resets to `NOT_STARTED` as above.

20. Definition of Done (per stage)

CDP_BUILD is done when: mart partition for `effective_month` exists; all `dbt tests` pass; `cdp_table_name` + `cdp_row_count` written; `source_total_row_count` reconciles within tolerance; `status='COMPLETED'`.

TRANSFORMATION is done when: Dataflow `JOB_STATE_DONE`; `_SUCCESS` + `manifest.json` present at `output_storage_path`; CSV row count reconciles to `cdp_row_count`; `output_storage_path` + `transformation_job_name` written; `status='COMPLETED'`.

21. Phased Build Plan

Phase	Deliverable	Exit criteria
0 — Infra	Projects, Cloud SQL instance, datasets (`mart`/`proc`), buckets, SAs, VPC-SC	`terraform apply` green; control schema migrated
1 — Control plane	`pipeline_run` + child tables, all Section 12 SQL, scheduler wrappers (no compute)	seed→claim→complete→error transitions pass integration tests
2 — CDP_BUILD	dbt project, Docker image, `CDP_BUILD_DBT_*` jobs, reconciliation test	one mart builds for a test month; tests gate; metadata updated
3 — TRANSFORMATION	Beam job, Flex Template, mapping template loader, `TRANSFORMATION_DATAFLOW_*`	CSV produced + verified for the same month end-to-end
4 — Reconciliation & observability	row-count chain, alerts, dashboard	breaches alert; dashboard shows a full run
5 — Hardening	retry/restart runbook, SLA tuning, dev→test→prod promotion, DR drill	failover + restart rehearsed; sign-off

Confirmed (GCP): Cloud SQL for PostgreSQL (control), BigQuery (mart + proc), Dataflow Flex Templates + Beam Java via Maven, Cloud Storage (output + config), Artifact Registry (images/templates), Workload Identity, VPC-SC. **Still to confirm:** processing-table TTL (default 14 days); child-FK shape (generated `stage_ref` column, Section 5.4 option a); driver `max_concurrency` value; whether the dbt container runs on Cloud Run Jobs or GKE. None change the overall design.

---

22. Non-Functional Requirements (NFRs)

The qualities the architecture must meet, with where each is satisfied. **Volume-dependent figures are marked *TBD* pending volumetrics** (see 22.1); the design choices that make the numbers achievable are already in place.

#	NFR	Requirement	How the design meets it
N1	**Availability / reliability**	The monthly run completes by its deadline; no single job failure loses state	Control store on **regional HA Cloud SQL** + PITR; all state externalised and **restartable**; atomic claim prevents double-runs; per-CDP isolation so one failure never blocks others
N2	**Performance / throughput**	Each stage within its duration budget; whole month within the delivery window	Partition-pruned reads (6.8), in-job chunking & autoscaling (7.5), set-based dbt build, BigQuery reservation for the window
N3	**Scalability**	Scale with more CDPs and growing data without redesign	Horizontal **fan-out by CDP** (4.4) bounded by `max_concurrency`; Dataflow autoscaling; BigQuery slots/reservation; build-only vs transform CDPs both supported
N4	**Capacity / volumetrics**	Sized for peak month (CDP count, rows, file sizes)	Sizing model in 16.2 keyed off `C = max_concurrency`; concrete figures ***TBD*** (22.1)
N5	**Security**	Least privilege, encryption, network isolation, auditability	Single scoped **`sa-cds-pipeline`** + Workload Identity (17); **VPC-SC** perimeter; GCP encryption at rest/in transit (CMEK optional); full audit logging; no secrets in code
N6	**Data quality / correctness**	Output is correct and complete or it is held	**Contract + DQ tests** gate the build (13); **completeness check** (5.7) and **reconciliation chain** (5.6, 12h); fixed CDP schema (13.1); output verified before release (R7)
N7	**Recoverability / DR**	Defined RTO/RPO; safe re-run and backfill	**RPO** ≈ Cloud SQL PITR window (near-zero for committed state); **RTO** via the restart runbook + idempotent re-run; monthly cadence gives generous recovery time; failover drill (Phase 5). Targets ***TBD***
N8	**Maintainability / extensibility**	Add or change a CDP with minimal change	**Config-driven**: catalogue, `cdp_registry`, mapping sheet (FDP→CDP), mapping template (CDP→file); generic template-driven Dataflow; artifacts promoted by **digest**
N9	**Observability**	Full visibility of run health and lineage	Section 18 — control-plane backbone, metrics, structured logs (`run_id`), dashboards, SLOs, Dynatrace, lineage in catalogue/child/manifest
N10	**Operability / supportability**	Operators can run, monitor and recover; clear ownership	Run Book + **support model & ServiceNow** routing (FDP issues to owning teams); per-CDP failure isolation
N11	**Cost efficiency**	Predictable, controllable spend	Partition pruning + `require_partition_filter` (R10); `proc` TTL; reservation vs on-demand; `max_concurrency` as the cost/throughput dial; cost monitoring
N12	**Auditability / compliance**	Provenance for every output; retention	`created/updated_timestamp` + optional `pipeline_run_history`; **lineage** (which FDP objects + which mapping versions produced an output) in the child table and `manifest.json`; retention policy ***TBD***
N13	**Portability**	Run on the chosen platform with limited lock-in	GCP-native by decision; standard engines (PostgreSQL, dbt, Apache Beam) ease portability; managed-service coupling is an accepted trade-off

22.1 Volumetrics & capacity (to confirm)

These drive N2–N4, N7 and the SLOs. **All values *TBD* — to be supplied** and then used to size the BigQuery reservation, Dataflow quota, Cloud SQL tier, and `max_concurrency`.

Dimension	Value (TBD)	Used to size
Number of CDPs (and build-only vs transform split)	*TBD*	fan-out, total runtime
Rows per mart / CDP (typical & peak)	*TBD*	BigQuery slots, Dataflow workers
FDP source sizes / partitions consumed	*TBD*	read time, reconciliation
Output file size & row count per CDP	*TBD*	shard count, transfer, retention
Monthly growth rate	*TBD*	headroom, reservation trend
Peak concurrent CDPs (`max_concurrency`)	*TBD*	quotas, connections
Delivery window (start after FDP cutoff → deadline)	*TBD*	end-to-end SLA

---

23. Service Levels — SLI, SLO, SLA

Definitions. An **SLI** (Service Level Indicator) is a *measured* signal of service health. An **SLO** (Service Level Objective) is the *internal target* for an SLI. An **SLA** (Service Level Agreement) is an *external commitment* to consumers, usually a subset of the SLOs with agreed consequences. SLIs are taken from the observability metrics (Section 18.2); SLOs drive the alert thresholds (Section 18.5); the SLA is the promise to the downstream (CDS / Mainframe) consumers.

23.1 SLIs and SLOs (internal targets)

Targets marked ***TBD*** depend on volumetrics (22.1) and consumer agreement.

Aspect	SLI (measured)	SLO (target)	Source / window
Monthly completion	% of in-scope CDPs `COMPLETED` by the deadline	e.g. **≥ 99% of months** all CDPs complete by the deadline (*TBD*)	control DB, per month
Build duration	`CDP_BUILD` end − start	**P95 ≤ *TBD***	control DB
Transform duration	`TRANSFORMATION` end − start	**P95 ≤ *TBD***	control DB
End-to-end freshness	time from all-FDP-ready to output delivered	**≤ *TBD* hours**	control DB + delivery
Correctness	reconciliation pass rate (source→mart→CSV)	**100% before any output is released**	reconciliation (5.6)
Completeness	required sources present & ready at gate	**100%** (no `MISSING/NOT_READY`)	completeness check (5.7)
Output delivery	file present + `_SUCCESS` at the agreed path/time	**≥ *TBD*%** of runs by deadline	GCS / MFT
Control-store availability	Cloud SQL uptime	**≥ 99.9%** (regional HA)	Cloud Monitoring
Stuck-run detection	time to detect a `STARTED` past budget	alert within ***TBD* min**	stuck detector (R1)

> **Error budget.** Each SLO implies an error budget (e.g. a 99% completion SLO allows ~1 month in 100 to miss). Repeated breaches trigger a review rather than silent acceptance.

23.2 SLA (external commitment)

The SLA is agreed with the downstream consumers (CDS / Mainframe) and the business; the figures are ***TBD*** until volumetrics and the delivery window are set. The committed shape:

- **Delivery:** the monthly output for each in-scope CDP is delivered to the agreed location by the agreed date/time (e.g. *working day N* of the month — ***TBD***).
- **Correctness:** delivered output is reconciled (counts agree end-to-end); incorrect output is **held, not delivered**.
- **Availability of the service:** the pipeline is operable each monthly cycle (control store **≥ 99.9%**).
- **Support response:** incident response/resolution by priority — **P1/P2/P3/P4 targets *TBD*** (Run Book support model, ServiceNow).

Upstream dependency (important). The end-to-end SLA depends on **FDP readiness by the agreed cutoff**. The pipeline SLA is therefore expressed **from all-FDP-ready** (or includes an explicit FDP cutoff); a late FDP consumes the consumer-facing window and is tracked and escalated to the owning team (R5, support model). This split keeps the pipeline accountable for what it controls.

23.3 Measurement & reporting

SLIs are computed from the control DB and platform metrics and surfaced on the **SLA / throughput dashboard** (18.4); SLOs are defined as **Dynatrace SLOs** (18.6) with Davis alerting; breaches raise ServiceNow incidents. SLA attainment is reported monthly.

---

24. Data Classification, PII & Access Governance

> **Placeholder — to be detailed in a later revision** (with the security / data-governance team).

Some FDP/CDP fields carry **PII**. At the architecture level, two principles apply and the specifics are deferred:

- **PII is classified and protected.** PII fields are identified (carried as a `classification` flag on the mapping sheet, Section 13.1) and protected in BigQuery; non-production uses masked or synthetic data. *Mechanism (e.g. column-level policy tags, restricted datasets, tokenisation), CMEK and DLP are to be confirmed.*
- **Human access is separate from the pipeline.** The pipeline runs as the machine account `sa-cds-pipeline` (Section 17); people never use it. Anyone needing to query the data directly (e.g. while building dbt models) obtains **least-privilege, time-bound, audited** access via the standard access-request workflow, with PII access granted separately. *The exact workflow, approvers and non-prod data approach are to be confirmed.*